МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО «СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
ИНСТИТУТ ПЕРСПЕКТИВНОЙ ИНЖЕНЕРИИ
ДЕПАРТАМЕНТ ЦИФРОВЫХ, РОБОТОТЕХНИЧЕСКИХ СИСТЕМ И ЭЛЕКТРОНИКИ

КУРСОВОЙ ПРОЕКТ

по дисциплине «Междисциплинарный проектный практикум»
на тему:
«Разработка детектора low-and-slow ботов по логам Apache»

Выполнил:
_____
студент ___ курса группы _____
направления подготовки 09.03.04 «Программная инженерия», профиль «Разработка и сопровождение программного обеспечения»

Руководитель проекта:
_____
(Ф. И. О., должность)

Проект допущен к защите _____ _____
(подпись руководителя) (дата)

Проект выполнен и защищен с оценкой _____ Дата защиты _____

Члены комиссии:
заведующий межинститутской базовой кафедрой департамента цифровых, робототехнических систем и электроники _____ Новикова Е. Н.
доцент межинститутской базовой кафедры департамента цифровых, робототехнических систем и электроники _____ Самойлов Ф. В.
доцент межинститутской базовой кафедры департамента цифровых, робототехнических систем и электроники _____ Альбекова З. М.
(подпись)

Ставрополь, 2026 г.

УТВЕРЖДАЮ
и.о. директора департамента цифровых, робототехнических систем и электроники
_____ Азаров И. В.
«___» _____ 2026 г.

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники
Направление подготовки 09.03.04 «Программная инженерия»
Направленность (профиль) «Разработка и сопровождение программного обеспечения»

ЗАДАНИЕ
на курсовой проект

студента _____
(фамилия, имя, отчество)
по дисциплине «Междисциплинарный проектный практикум»

1. Тема проекта: «Разработка детектора low-and-slow ботов по логам Apache».
2. Цель проекта: разработка программной системы, выявляющей low-and-slow ботов по журналам веб-сервера Apache с помощью эвристических правил и методов машинного обучения, включая нейронную сеть.
3. Задачи:
   3.1 Проанализировать предметную область и существующие решения, сформулировать требования.
   3.2 Спроектировать архитектуру, модель данных и ML-пайплайн, выбрать технологический стек.
   3.3 Реализовать правила, извлечение признаков, модели поиска аномалий, REST API и веб-интерфейс.
   3.4 Провести тестирование и оценить качество моделей и эффективность решения.
4. Перечень подлежащих разработке вопросов:
   а) теоретической части: характеристика low-and-slow ботов и признаков их обнаружения; обзор аналогов; постановка технического задания;
   б) практической части: обоснование выбора технологического стека и метода машинного обучения; проектирование и программная реализация системы; тестирование и оценка качества.
5. Исходные данные: журнал веб-сервера Apache интернет-магазина в формате combined; документация поисковых систем по проверке роботов; рекомендованная литература по машинному обучению.
6. Рекомендуемая литература: Гудфеллоу Я. и др. Глубокое обучение; Рашка С. Python и машинное обучение; Мартин Р. Чистая архитектура; документация scikit-learn, PyTorch, FastAPI.
7. Контрольные сроки представления отдельных разделов курсового проекта:
   25 % – составление плана работы, подбор литературы «___» _____ 2026 г.
   50 % – предоставление первого раздела «___» _____ 2026 г.
   75 % – предоставление второго раздела «___» _____ 2026 г.
   100 % – предоставление работы на отзыв «___» _____ 2026 г.
8. Срок защиты студентом курсового проекта «___» _____ 2026 г.

Дата выдачи задания «___» _____ 2026 г.

Руководитель курсового проекта _____ _____
(личная подпись) (инициалы, фамилия)

Задание принял к исполнению студент очной формы обучения ___ курса группы
_____ _____ _____

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 5
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ФОРМУЛИРОВАНИЕ ТРЕБОВАНИЙ 7
1.1 Характеристика предметной области и постановка проблемы 7
1.2 Обзор и анализ существующих решений 8
1.3 Целевая аудитория и сценарии использования 9
1.4 Формулировка требований к программному обеспечению 11
1.5 Техническое задание 12
Выводы по главе 1 13
2 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ И МОДЕЛИ ДАННЫХ, ВЫБОР ТЕХНОЛОГИЧЕСКОГО СТЕКА 14
2.1 Архитектура системы 14
2.2 Проектирование данных 16
2.3 Проектирование ML-пайплайна 18
2.4 Выбор технологического стека 21
2.5 Проектирование пользовательского интерфейса 21
2.6 Структура проекта 22
Выводы по главе 2 22
3 РЕАЛИЗАЦИЯ ПРОГРАММНОГО ПРОДУКТА 23
3.1 Организация процесса разработки 23
3.2 Разбор журнала и анализ User-Agent 23
3.3 Эвристические правила 24
3.4 Реализация ML-компонента 25
3.5 Реализация бэкенда 27
3.6 Реализация фронтенда 29
3.7 Принципы качественного кода 29
Выводы по главе 3 30
4 ТЕСТИРОВАНИЕ И ОЦЕНКА РЕЗУЛЬТАТОВ 31

4.1 Тест-план и автоматические тесты 31
4.2 Тестирование ML-компонента 31
4.3 Анализ ошибок 34
4.4 Функциональное тестирование 35
4.5 Производительность и нагрузочное тестирование 36
4.6 Найденные и исправленные ошибки 37
4.7 Оценка эффективности 37
Выводы по главе 4 38
ЗАКЛЮЧЕНИЕ 39
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 41
ПРИЛОЖЕНИЕ А (обязательное) Листинги программного кода 44
ПРИЛОЖЕНИЕ Б (обязательное) Диаграммы классов и развёртывания 52
ПРИЛОЖЕНИЕ В (обязательное) Дополнительные результаты тестирования 54

ВВЕДЕНИЕ

Значительная часть трафика современных сайтов создаётся не людьми, а программами. По оценке компании Imperva, в 2023 году автоматизированные запросы составили почти половину всего интернет-трафика, причём около трети приходилось на «плохих» ботов – парсеров, сборщиков цен, средств перебора и мошенничества [1]. Для интернет-магазина такие боты означают копирование каталога и цен конкурентами, искажение веб-аналитики и лишнюю нагрузку на сервер. Сообщество OWASP выделяет эти действия в отдельный класс автоматизированных угроз веб-приложениям [2].

Простых ботов, которые отправляют сотни запросов в минуту, легко отсечь ограничением частоты. Поэтому всё чаще используются **low-and-slow боты**: они делают один запрос в десятки секунд или минут, распределяют работу по многим IP-адресам и подделывают User-Agent браузеров и поисковых роботов. По интенсивности такой бот неотличим от человека, и пороговые средства защиты его не видят. Обнаружить его можно только по совокупности косвенных признаков поведения, что делает задачу естественной областью применения методов поиска аномалий и машинного обучения.

Объект исследования – веб-трафик интернет-магазина, зафиксированный в журналах веб-сервера Apache. **Предмет исследования** – методы и программные средства обнаружения low-and-slow ботов по журналам веб-сервера на основе эвристических правил и моделей машинного обучения.

Цель проекта – разработать программную систему, которая по журналу Apache находит IP-адреса low-and-slow ботов, объясняет причину решения и оценивает аномальность поведения каждого IP с помощью моделей машинного обучения, в том числе нейронной сети.

Для достижения цели поставлены задачи, соответствующие главам записки:

1) проанализировать предметную область и существующие решения, определить сценарии использования, сформулировать

функциональные и нефункциональные требования и техническое задание (глава 1);

2) спроектировать архитектуру системы, модель данных и ML-пайплайн, обосновать выбор технологий (глава 2);
3) реализовать разбор журнала, эвристические правила, извлечение признаков, модели поиска аномалий (включая нейросетевой автоэнкодер), REST API и веб-интерфейс (глава 3);
4) протестировать систему, оценить качество моделей и эффективность решения по сравнению с исходным прототипом (глава 4).

Практическая значимость. Система работает офлайн по уже существующим журналам и не требует проксирования трафика через сторонние сервисы. Администратор получает список подозрительных IP с объяснением причин и может передать его фаерволу или системе мониторинга через REST API. Модели обучаются на трафике конкретного сайта, поэтому решение переносится на другие проекты без ручной настройки порогов.

Методы и средства разработки: анализ журналов веб-сервера, проверка поисковых роботов через DNS, эвристики правдоподобия User-Agent, статистический анализ интервалов, методы поиска аномалий (Isolation Forest, One-Class SVM, Local Outlier Factor, автоэнкодер); язык Python, библиотеки pandas, scikit-learn, PyTorch, FastAPI, СУБД SQLite, система контроля версий Git, тестирование pytest.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ФОРМУЛИРОВАНИЕ ТРЕБОВАНИЙ

1.1 Характеристика предметной области и постановка проблемы

Веб-сервер Apache записывает каждый HTTP-запрос в журнал доступа. Наиболее распространён формат combined [3]: IP-адрес клиента, время запроса с часовым поясом, строка запроса (метод, путь, протокол), код ответа, размер ответа, заголовок Referer (откуда пришёл посетитель) и заголовок User-Agent (чем посетитель представился). Пример строки:

```
203.0.113.7 - - [08/Sep/2026:09:17:29 +0300] "GET /catalog/pens
HTTP/2.0"
200 45512 "https://shop.example/" "Mozilla/5.0 (Windows NT 10.0;
...) Chrome/150.0.0.0"
```

Других источников о клиенте у журнала нет: всё, что известно о посетителе, – это последовательность его запросов во времени и то, чем он представился. Поэтому детектор должен опираться на поведение, а не на одиночный запрос.

Источником данных для проекта послужил журнал доступа интернет-магазина канцелярских товаров за 8 сентября 2026 года: 395 000 строк за 2 ч 40 мин, 6 706 уникальных IP-адресов, 96 % ответов с кодом 200. Журнал содержит реальные IP-адреса и историю посещений покупателей, поэтому в публичный репозиторий проекта он не включается; для примеров и тестов создан полностью синтетический журнал вымышленного магазина.

В журнале встречаются три группы клиентов: люди, легитимные поисковые роботы (Googlebot, YandexBot, Bingbot) и прочие боты. Поисковые системы рекомендуют проверять своих роботов двойным DNS-запросом: обратный запрос (PTR) по IP должен вернуть имя в домене поисковика, а прямой запрос по этому имени – тот же IP [4, 5]. Проверка исходного журнала показала масштаб подделки: из 2 955 IP, представлявшихся поисковыми роботами, подтвердить удалось только 521, а 2 434 IP (82 %) оказались поддельными – у 1 535 нет PTR-записи вовсе, ещё 899 разрешаются в домены домашних провайдеров.

Low-and-slow бот специально держит интенсивность ниже порогов защиты, но его поведение отличается от поведения человека рядом признаков (таблица 1.1).

Таблица 1.1 – Признаки, отличающие low-and-slow бота от человека

Признак	**Человек**	**Low-and-slow бот**
Интервалы между визитами	нерегулярные	почти равные (таймер в скрипте) или случайные без «пачек»
Загрузка статики (изображения, CSS, JS)	вместе со страницей	часто отсутствует
User-Agent	реальный, согласованный	подделан, внутренне противоречив, «поисковик» без подтверждения DNS
Распределение по IP	один IP – один посетитель	много IP одной подсети в одно время
Набор страниц	навигация, поиск, корзина	однотипный обход карточек и категорий
Referer	внутренний переход	пустой или одинаковый

Таким образом, **проблема** состоит в том, что: пороговые средства защиты не замечают медленных ботов; одиночный признак (например, User-Agent) легко подделать; подделка поисковых роботов массовая и требует DNS-проверки; комбинация слабых признаков поведения требует автоматического анализа, а не ручного просмотра журнала. Исходный прототип на Python решал задачу только набором правил и, как показал анализ (раздел 3.2), содержал ошибку, из-за которой часть проверок User-Agent не выполнялась.

1.2 Обзор и анализ существующих решений

Для сравнения выбраны четыре широко используемых решения: средство блокировки по журналам fail2ban [6], коллективная система обнаружения атак CrowdSec [7], облачный сервис Cloudflare Bot Management [8] и анализатор журналов GoAccess [9]. Результаты сравнения приведены в таблице 1.2.

Таблица 1.2 – Сравнение аналогов

Критерий	**fail2ban**	**CrowdSec**	**Cloudflare Bot Mgmt**	**GoAccess**	**Разрабатываемая система**
Работа по журналам Apache	да	да	нет (прокси)	да	да
Поиск медленных ботов	нет	частично	да	нет	да
Проверка поисковиков по DNS	нет	частично	да	нет	да
Правдоподобие User-Agent	нет	нет	да	нет	да
Машинное обучение	нет	нет	да (закрытая модель)	нет	да (нейросеть и 3 модели)
Объяснение решения	регулярное выражение	сценарий	числовой балл	–	правило + признаки
Работа без передачи данных третьим лицам	да	да	нет	да	да
Стоимость	бесплатно	бесплатно / платные функции	тариф Enterprise	бесплатно	бесплатно

fail2ban и CrowdSec блокируют клиентов по превышению частоты событий и шаблонам атак; медленные боты под эти пороги не попадают. GoAccess только визуализирует статистику. Cloudflare Bot Management решает задачу, но требует пропускать весь трафик через инфраструктуру Cloudflare, стоит дорого и не раскрывает логику решения.

Конкурентные преимущества разрабатываемой системы: офлайн-анализ существующих журналов без передачи данных; сочетание объяснимых правил и ML-модели, которая находит ботов без явного признака; объяснение каждого решения главными признаками; обучение под конкретный сайт; открытый код и бесплатность.

1.3 Целевая аудитория и сценарии использования

Пользователи системы: **администратор сайта**, который ищет и блокирует ботов; **аналитик безопасности**, который обучает модели и настраивает пороги; **внешняя система** (фаервол, SIEM), получающая список IP по API. Внешним актёром выступают также DNS-серверы, к которым система обращается при

проверке поисковых роботов. Диаграмма вариантов использования показана на рисунке 1.1.

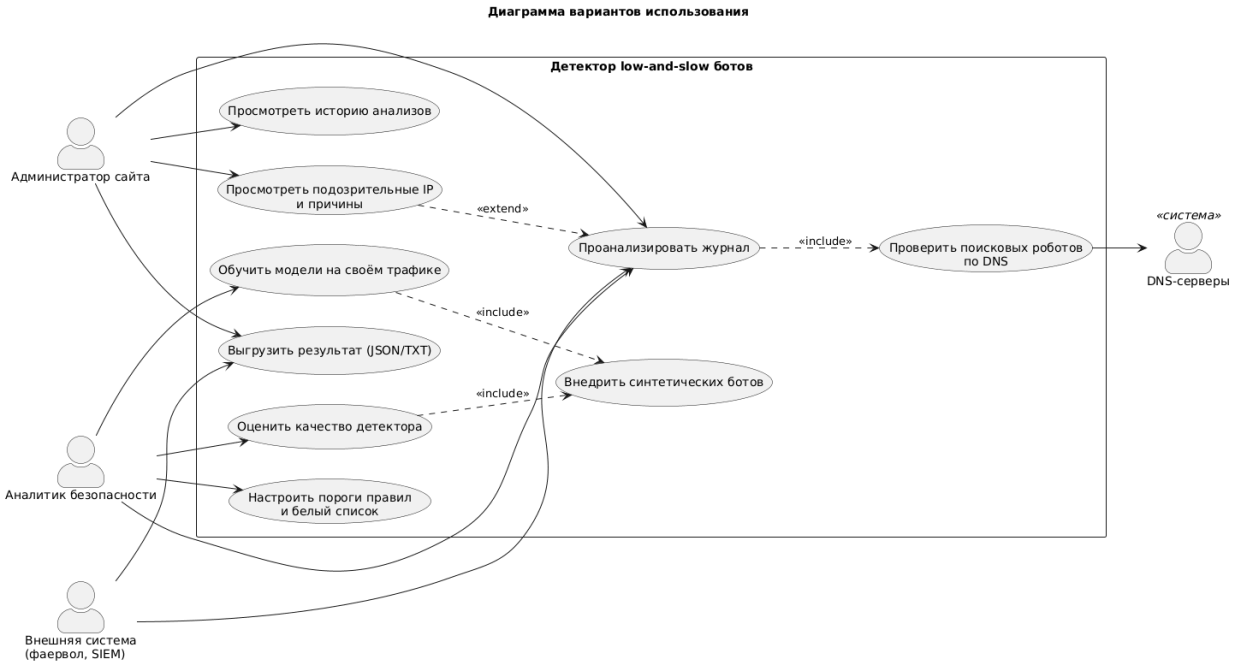


Рисунок 1.1 – Диаграмма вариантов использования

Ключевые сценарии использования:

1) **Проанализировать журнал.** Администратор загружает access.log, система выдаёт список подозрительных IP с причинами
2) **Обучить модель на своём трафике.** Аналитик запускает обучение на журнале обычного дня, модели сохраняются для последующих анализов
3) **Оценить качество детектора.** В журнал внедряются синтетические боты, считаются метрики и строятся графики
4) **Настроить пороги.** Аналитик меняет окно подсети, порог регулярности интервалов, белый список сетей
5) **Получить результат по API.** Внешняя система отправляет журнал и получает JSON со списком IP для блокировки
6) **Вернуться к прошлому анализу.** Администратор открывает результат из истории и смотрит, как менялось решение по IP

Требования пользователей в форме user stories: «как администратор, я хочу видеть, почему IP признан ботом, чтобы не заблокировать покупателя»; «как

аналитик, я хочу получать оценку аномальности от модели, чтобы находить ботов, которые не попадают ни под одно правило»; «как разработчик интеграции, я хочу JSON-API, чтобы автоматически передавать список IP фаерволу».

1.4 Формулировка требований к программному обеспечению

Функциональные требования перечислены в таблице 1.3, нефункциональные – в таблице 1.4.

Таблица 1.3 – Функциональные требования

№	Требование
ФТ-1	Разбор журнала Apache в формате combined (IP, время, метод, путь, статус, размер, Referer, User-Agent), подсчёт пропущенных строк
ФТ-2	Проверка заявленных поисковых роботов (Google, Yandex, Bing) через reverse + forward DNS с кэшированием результатов
ФТ-3	Выявление невозможных и противоречивых User-Agent (несуществующие версии, несовместимые ОС и браузер, средства автоматизации)
ФТ-4	Выявление аномально равномерных интервалов между визитами
ФТ-5	Выявление групп IP одной подсети, активных в одном временном окне
ФТ-6	Выявление смены браузера или ОС на одном IP
ФТ-7	Извлечение поведенческих признаков для каждого IP (не менее 15)
ФТ-8	Обучение моделей поиска аномалий, сохранение и загрузка обученных моделей
ФТ-9	Оценка аномальности каждого IP и итоговое решение по правилам и модели
ФТ-10	Генерация синтетических low-and-slow ботов для оценки качества по известной разметке
ФТ-11	Расчёт метрик (Precision, Recall, F1, ROC-AUC), матрицы ошибок и графиков
ФТ-12	Выгрузка результатов в TXT и JSON
ФТ-13	REST API: загрузка журнала, запуск анализа, получение результатов
ФТ-14	Веб-страница: загрузка файла, таблица подозрительных IP с фильтрами, графики
ФТ-15	Белый список подсетей, исключаемых из проверки
ФТ-16	История анализов: повторное открытие результата, удаление, история решений по одному IP

Таблица 1.4 – Нефункциональные требования

№	Требование
НФТ-1	Производительность: журнал 400 тыс. строк обрабатывается не дольше 60 с без учёта DNS-запросов

№	Требование
НФТ-2	Точность: на тестовой выборке с синтетическими ботами F1 ≥ 0,85 и Precision ≥ 0,9 итогового решения
НФТ-3	Объяснимость: для каждого подозрительного IP указываются сработавшие правила и признаки с наибольшим отклонением
НФТ-4	Воспроизводимость: фиксированные начальные значения генераторов случайных чисел, кэш DNS, сохранённые модели
НФТ-5	Конфиденциальность: данные не покидают сервер, в репозиторий попадают только синтетические данные
НФТ-6	Переносимость: Python 3.10+, Windows и Linux, зависимости в requirements.txt
НФТ-7	Качество кода: PEP 8, модульная структура, unit-тесты ключевых модулей, непрерывная интеграция

Технологические ограничения: язык Python, библиотеки машинного обучения из стека, рекомендованного методическими указаниями. Требования к интерфейсу: одна страница, на которой выполняются все сценарии администратора; навигация без перезагрузки страницы; понятные подписи на русском языке.

1.5 Техническое задание

Техническое задание составлено в упрощённом виде по ГОСТ 34.602-89 [10].

- **Наименование:** «Детектор low-and-slow ботов по журналам Apache» (lsdetector);
- **Назначение:** офлайн-выявление медленных ботов-парсеров и поддельных поисковых роботов по журналам доступа;
- **Цели создания:** повысить полноту обнаружения ботов по сравнению с прототипом на правилах при сохранении высокой точности и объяснимости;
- **Входные данные:** журнал Apache в формате combined (текст, UTF-8); параметры правил; обученные модели;
- **Выходные данные:** список IP с причинами, оценками моделей и итоговым решением (TXT, JSON, веб-таблица); метрики и графики качества;

– **Состав системы:** модуль разбора, модуль правил, модуль признаков, ML-модуль, модуль оценки качества, хранилище истории, REST API, веб-интерфейс, интерфейс командной строки;
– **Требования к системе:** функциональные ФТ-1…ФТ-16 и нефункциональные НФТ-1…НФТ-7 (таблицы 1.3, 1.4);
– **Стадии работ:** анализ и требования; проектирование; реализация ядра, ML-компонента, API и интерфейса; тестирование и оценка; оформление документации – в соответствии с процессами жизненного цикла ГОСТ Р ИСО/МЭК 12207-2010 [11];
– **Порядок приёмки:** прохождение автоматических тестов, достижение целевых метрик на тестовой выборке, демонстрация CLI, REST API и веб-страницы.

Выводы по главе 1

Проанализирована предметная область: low-and-slow боты не превышают пороги частоты, поэтому их нужно искать по совокупности признаков поведения. На реальном журнале магазина показано, что 82 % IP, представлявшихся поисковыми роботами, – подделка. Сравнение аналогов показало, что открытые инструменты ловят только быстрых ботов, а коммерческие требуют проксирования трафика; это определяет нишу разрабатываемой системы. Сформулированы 6 сценариев использования, 16 функциональных и 7 нефункциональных требований, составлено техническое задание.

2 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ И МОДЕЛИ ДАННЫХ, ВЫБОР ТЕХНОЛОГИЧЕСКОГО СТЕКА

2.1 Архитектура системы

Выбран архитектурный стиль **клиент-серверного модульного монолита**: одно ядро в виде Python-пакета и два тонких интерфейса к нему – REST API и командная строка. Для офлайн-анализа журналов микросервисы избыточны: в системе нет частей, которые нужно масштабировать независимо, а один процесс проще развернуть администратору. Архитектура описана в нотации C4 [12] на трёх уровнях: контекст, контейнеры и компоненты.

На диаграмме контекста (рисунок 2.1) показано окружение системы: веб-сервер Apache создаёт журнал, администратор и аналитик работают с системой, DNS используется для проверки поисковых роботов, фаервол или SIEM получают результат.

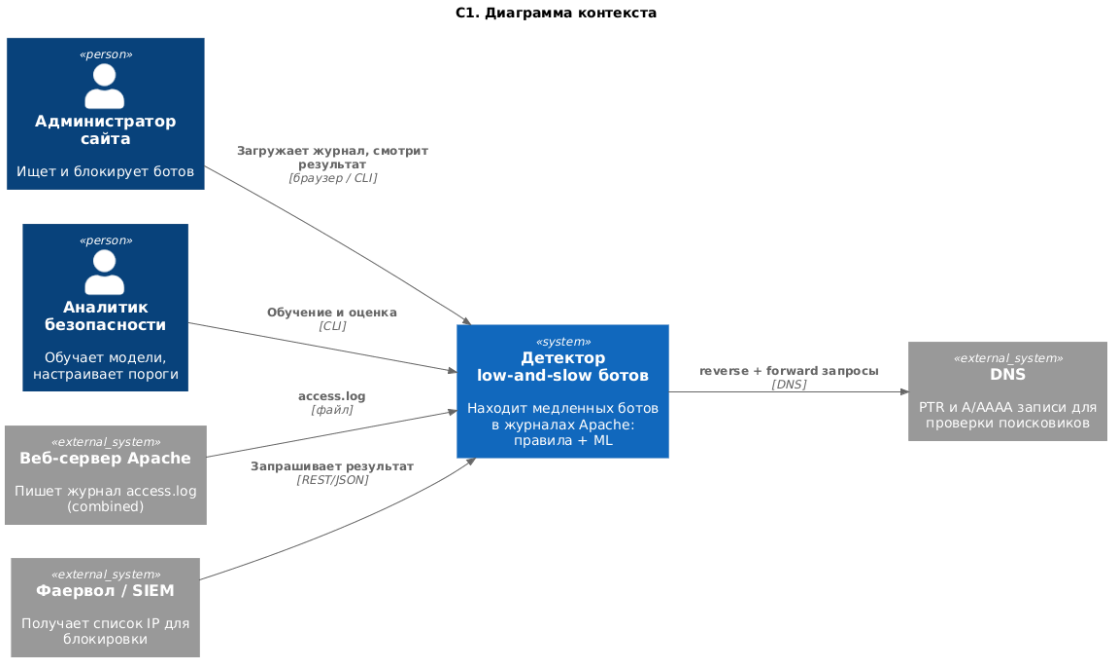


Рисунок 2.1 – Диаграмма контекста (C4, уровень 1)

Диаграмма контейнеров (рисунок 2.2) раскрывает внутреннее устройство системы: веб-страница обращается к REST API, API и командная строка

вызывают ядро lsdetector, история анализов хранится в SQLite, обученные модели и кэш DNS – в файлах.

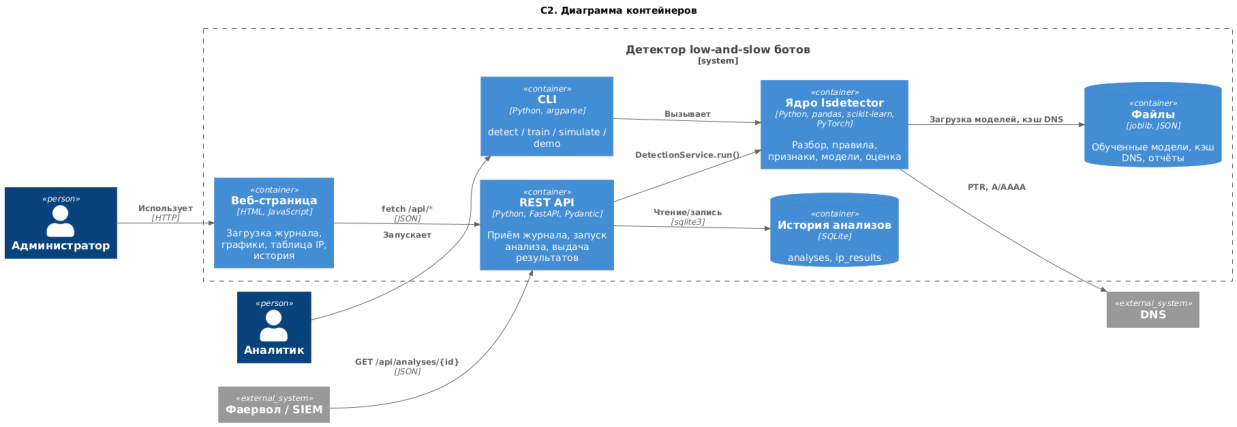


Рисунок 2.2 – Диаграмма контейнеров (C4, уровень 2)

Компоненты ядра (рисунок 2.3, таблица 2.1) зависят только от «нижележащих» модулей: разбор журнала ничего не знает о правилах, правила – о моделях, а фасад DetectionService собирает их вместе.

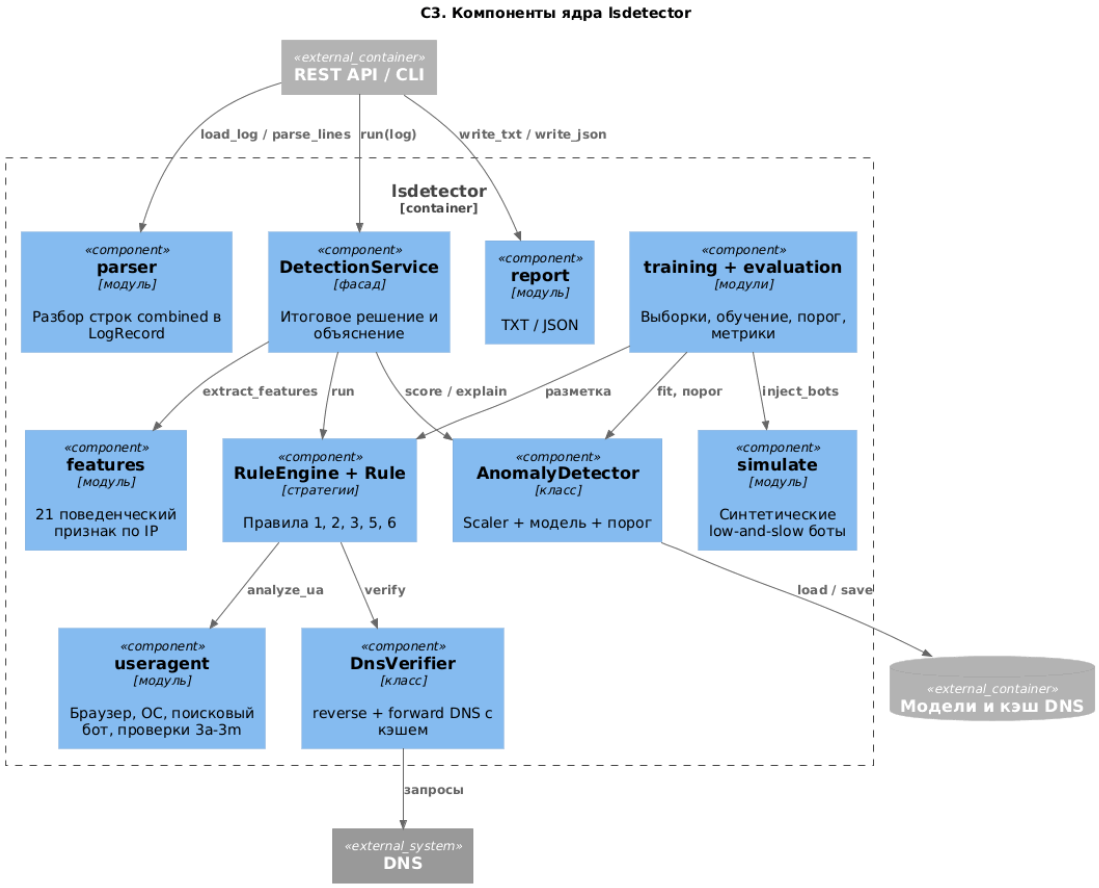


Рисунок 2.3 – Диаграмма компонентов ядра (C4, уровень 3)

Таблица 2.1 – Компоненты ядра lsdetector

Компонент	Назначение
parser	разбор строк журнала в записи LogRecord, группировка по IP, статистика пропусков
useragent	определение браузера, ОС и поискового робота, 13 проверок правдоподобия User-Agent
DnsVerifier	reverse + forward DNS-проверка поисковых роботов с кэшем
RuleEngine, Rule	эвристические правила 1, 2, 3, 5, 6 и движок их применения
features	21 поведенческий признак для каждого IP
AnomalyDetector	модель поиска аномалий вместе с масштабированием признаков и порогом
DetectionService	фасад: итоговое решение по правилам и модели, объяснение
simulate, demo	генерация синтетических ботов и демонстрационного журнала
training, evaluation	выборки, обучение, подбор порога, метрики, графики, разбор ошибок
report	выгрузка результатов в TXT и JSON

2.2 Проектирование данных

Логическая модель. Журнал состоит из записей LogRecord (IP, время в UTC, смещение часового пояса, метод, путь, статус, размер, Referer, User-Agent). Записи группируются по IP; для каждой строки User-Agent вычисляются сведения UAInfo (робот, браузер, ОС, коды причин правила 3); для IP – вектор признаков и итоговое решение IPVerdict (правила, оценки моделей, флаг «бот», причина, главные признаки).

Хранение. Журнал обрабатывается в памяти за один проход, поэтому сырые записи в базу данных не сохраняются. В базу попадает история анализов: сводка прогона и решения по каждому IP. Это позволяет вернуться к прошлому результату, переслать ссылку на него и посмотреть, как менялось решение по одному IP между прогонами.

Выбор СУБД – SQLite. Она встроена в Python (модуль sqlite3), не требует отдельного сервера, база хранится в одном файле; нагрузка – один администратор и редкие запросы внешних систем. Режим журнала WAL [13] позволяет нескольким процессам веб-сервера писать в базу по очереди. Доступ

к данным изолирован в классе AnalysisRepository, поэтому переход на PostgreSQL затронет только этот класс.

Физическая модель состоит из двух таблиц (рисунок 2.4, таблица 2.2); DDL-скрипт приведён в листинге А.5 приложения А.

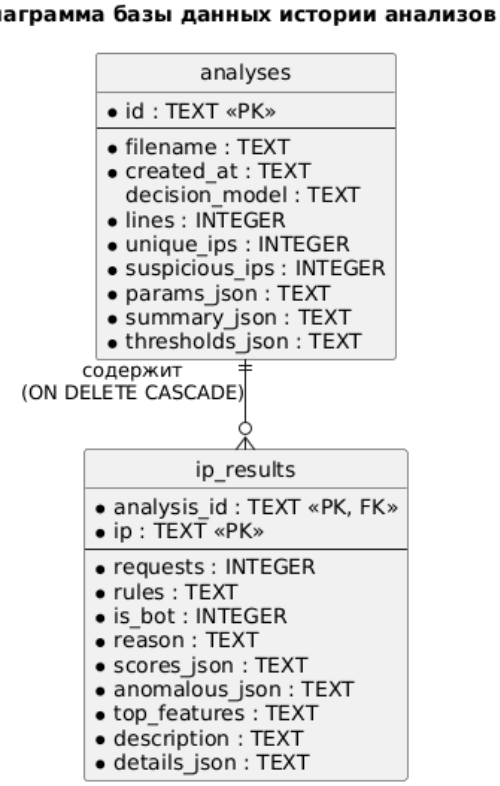


Рисунок 2.4 – ER-диаграмма базы данных истории анализов

Таблица 2.2 – Таблицы базы данных

Таблица	Ключ	Содержимое
analyses	id (PK)	один загруженный журнал: имя файла, дата, модель, число строк, IP и ботов, параметры запуска, сводка и пороги моделей (JSON)
ip_results	(analysis_id, ip) (PK); analysis_id → analyses.id, ON DELETE CASCADE	решение по IP: число запросов, правила, флаг «бот», причина, оценки моделей, главные признаки, описание

Индексы созданы по дате анализа, по паре (analysis_id, is_bot) для выборки только ботов и по IP для истории одного адреса. Вложенные структуры – оценки разных моделей и подробности правил – хранятся в JSON-полях: их состав зависит от набора моделей и правил, а поиск по ним не требуется.

2.3 Проектирование ML-пайплайна

ML-компонент решает задачу поиска аномалий: модель изучает поведение обычных посетителей и оценивает, насколько сильно поведение каждого IP от него отличается [14]. Пайплайн обучения показан на рисунке 2.5.

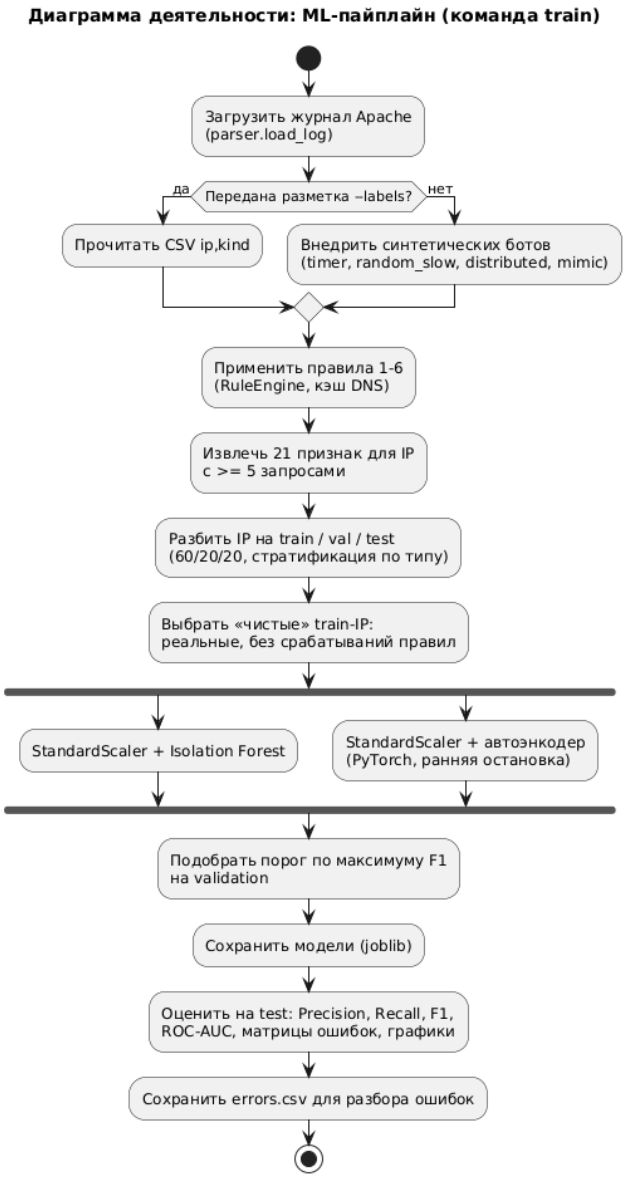


Рисунок 2.5 – Диаграмма деятельности ML-пайплайна

Сбор и предобработка данных. Источник – журнал Apache. Нераспознанные строки отбрасываются, IP нормализуются, записи каждого IP сортируются по времени. IP с менее чем пятью запросами модели не оцениваются: по нескольким запросам поведение не определить, такие IP

проверяют только правила. В исходном журнале таких «содержательных» IP – 2 254.

Выделение признаков. Для каждого IP рассчитывается 21 признак (таблица 2.3). Счётчики логарифмируются (log(1 + x)), затем все признаки стандартизируются (StandardScaler), а значения обрезаются до ±8 стандартных отклонений, чтобы один экстремальный IP не искажал обучение.

Таблица 2.3 – Поведенческие признаки IP

Группа	Признаки
Интенсивность	число запросов, число визитов (запросы с разрывом до 1 с склеиваются), длительность активности, запросов в визите
Регулярность	медианный интервал между визитами, коэффициент вариации интервалов, энтропия интервалов, доля самого длинного перерыва
Состав запросов	доля статики, доли ответов 4xx, 5xx и 3xx, доля POST, доля запросов с параметрами, средний размер ответа, доля уникальных путей
Источник перехода	доля запросов без Referer, доля Referer с чужих сайтов
Клиент и сеть	число разных User-Agent, число пар (браузер, ОС), число IP той же подсети /24 в журнале

Разметка. Достоверной разметки «бот/человек» для реального журнала нет, а ручная разметка тысяч IP недостоверна. Поэтому применён подход с внедрением синтетических аномалий: генератор добавляет в журнал ботов с известным поведением четырёх типов (таблица 2.4). Пути страниц, размеры ответов и User-Agent берутся из того же журнала, чтобы боты не выделялись признаками, которых у них не может быть. IP ботов выделяются из диапазона 198.18.0.0/15, зарезервированного для тестирования, и не пересекаются с реальными. В исходный журнал внедряется 516 ботов.

Таблица 2.4 – Типы синтетических low-and-slow ботов

Тип	Поведение
timer	визиты по таймеру с периодом 30–400 с и небольшим разбросом, только HTML-страницы, без Referer
random_slow	случайные (пуассоновские) интервалы со средним 1–10 мин, последовательный обход каталога без статики
distributed	группа из 6–12 IP одной подсети /24, каждый делает 5–15 запросов в общем окне 10–60 мин
mimic	имитация браузера: подгружает 1–4 статических файла на страницу и ставит внутренний Referer

Разбиение выборок. IP делятся на обучающую, валидационную и тестовую выборки в пропорции 60/20/20 со стратификацией по типу бота и срабатыванию правил. Модели обучаются без учителя только на «чистых» IP обучающей выборки – реальных и без срабатывания надёжных правил. Порог оценки подбирается на валидационной выборке по максимуму F1, качество измеряется на тестовой.

Выбор модели. Рассматриваются четыре метода поиска аномалий без учителя: Isolation Forest – ансамбль случайных деревьев, изолирующий аномалии за малое число разбиений [15]; One-Class SVM – граница вокруг нормальных данных в пространстве RBF-ядра [16]; Local Outlier Factor – сравнение локальной плотности точки с плотностью соседей [17]; **автоэнкодер** – нейронная сеть, которая сжимает вектор признаков до узкого скрытого слоя и восстанавливает его обратно [18, 19]. Автоэнкодер обучается только на обычных посетителях, поэтому поведение ботов восстанавливает хуже, и ошибка восстановления служит оценкой аномальности; ошибка по каждому признаку объясняет, чем IP отличается от нормы. Выбор основной модели и её архитектуры выполнен экспериментально (раздел 4.2).

Итоговое решение. IP признаётся ботом, если сработало надёжное правило (1, 3, 5 или 6) **или** оценка модели выше порога. Правило 2 (смена браузера на одном IP) часто срабатывает на пользователей за общим NAT, поэтому считается слабым сигналом и только попадает в отчёт.

2.4 Выбор технологического стека

Сравнение альтернатив и обоснование выбора приведены в таблице 2.5.

Таблица 2.5 – Выбор технологий

Задача	Выбрано	Альтернативы	Обоснование
Язык	Python 3.11	Go, Java	экосистема ML, рекомендован методическими указаниями; скорость разбора достаточна (НФТ-1)
Табличные данные	pandas, numpy	polars	стандарт де-факто, совместим со scikit-learn [20]
Классические модели	scikit-learn	PyOD	Isolation Forest, One-Class SVM, LOF, метрики и разбиение в одной библиотеке [21, 22]
Нейросеть	PyTorch	TensorFlow / Keras	простой собственный цикл обучения с ранней остановкой и доступом к ошибке по признакам; работа на CPU [23, 24]
Сохранение моделей	joblib + state_dict	pickle, ONNX	эффективная сериализация массивов; ONNX не нужен – инференс в том же Python
REST API	FastAPI + Pydantic	Flask, Django	автоматическая документация Swagger, валидация параметров, пул потоков [25]
Веб-интерфейс	HTML + JavaScript	React, Vue	одна страница, без сборки и внешних зависимостей, работает без интернета
СУБД	SQLite	PostgreSQL, MongoDB	встроена в Python, без отдельного сервера; нагрузка одного администратора
Графики	matplotlib, SVG	plotly	статичные изображения для отчёта; в браузере – без библиотек
Тесты и стиль	pytest, ruff	unittest, flake8	фикстуры и параметризация; ruff объединяет линтер и сортировку импортов
CI и диаграммы	GitHub Actions, PlantUML	GitLab CI, draw.io	репозиторий на GitHub; текстовые исходники диаграмм в Git, единая нотация UML/C4

2.5 Проектирование пользовательского интерфейса

Интерфейс администратора – одна веб-страница, сверху вниз: форма загрузки журнала с выбором модели и параметрами правил; таблица истории анализов; карточки сводки (строк, IP, заявленных и подтверждённых поисковиков, подозрительных IP, времени); два графика – причины срабатывания и гистограмма оценок модели с порогом; таблица IP с поиском, фильтрами («только боты», «найдены только моделью», по номеру правила),

сортировкой и кнопкой выгрузки JSON. Основной сценарий: выбрать файл → «Анализировать» → изучить сводку и графики → отфильтровать таблицу → скачать JSON. Прошлые анализы открываются из таблицы истории; ссылка вида /#id=<идентификатор> открывает конкретный результат. Требования к эргономике: всё на одной странице, без перезагрузок, адаптивная вёрстка, поддержка тёмной темы.

2.6 Структура проекта

Исходный код организован в каталоги:

```
lsdetector/     ядро: разбор, правила, признаки, модели, оценка, CLI
  models/       Isolation Forest, автоэнкодер, One-Class SVM, LOF
api/            REST API (FastAPI), схема и хранилище SQLite
web/            веб-страница (HTML, CSS, JavaScript)
models/         обученные модели (pretrained, demo)
data/demo/      синтетический журнал с разметкой ботов
scripts/        эксперименты с моделями, нагрузочный тест
tests/          unit- и интеграционные тесты (pytest)
docs/           документация по этапам, UML-диаграммы, результаты
```

Выводы по главе 2

Выбран архитектурный стиль модульного монолита с REST API и CLI, архитектура описана диаграммами C4 трёх уровней. Спроектирована модель данных: логическая модель обработки в памяти и физическая модель SQLite для истории анализов. Разработан ML-пайплайн: 21 поведенческий признак, разметка внедрением синтетических ботов четырёх типов, обучение без учителя на чистых данных, подбор порога на валидации, итоговое решение «надёжное правило или модель». Обоснован выбор технологий, спроектирован интерфейс и структура проекта.

3 РЕАЛИЗАЦИЯ ПРОГРАММНОГО ПРОДУКТА

3.1 Организация процесса разработки

Работа планировалась списком задач по этапам (Kanban-доска с колонками «требования – проектирование – ядро – ML – API и интерфейс – тесты – документация»). Исходный код хранится в Git и опубликован на GitHub: https://github.com/Gregory7635/low-and-slow-detector. Применена стратегия ветвления main – develop – feature: стабильные версии в ветке main (версии 0.1.0 и 0.2.0 помечены тегами), интеграция в develop, каждый этап – в отдельной ветке feature/* (docs, core, ml, api, tests, report-docs, model-comparison, storage, uml), которые сливаются без перемотки (merge --no-ff), чтобы история этапов оставалась видимой.

Инструменты: редактор Visual Studio Code, менеджер зависимостей pip (requirements.txt, requirements-dev.txt), настройки проекта в pyproject.toml, линтер ruff, тесты pytest. На каждый push в GitHub Actions автоматически запускаются линтер и все тесты.

3.2 Разбор журнала и анализ User-Agent

Строка журнала разбирается одним регулярным выражением; время переводится в UTC вручную, с учётом часового пояса, IP-адреса нормализуются модулем ipaddress (в том числе сокращённая запись IPv6). Нераспознанные строки не прерывают работу, а учитываются в статистике разбора. Журнал из 395 000 строк разбирается примерно за 11 с.

Модуль useragent определяет поискового робота, браузер и ОС и выполняет 13 проверок правдоподобия User-Agent (таблица 3.1). Проверки опираются на таблицу актуальных версий браузеров и известные правила формирования строки: например, Chrome с версии 101 на настольных системах и 110 на Android урезает User-Agent до вида «Chrome/120.0.0.0» и «Android 10; K» [26], а номер сборки Chrome однозначно связан с мажорной версией. Результат анализа кэшируется по строке User-Agent (functools.lru_cache).

Таблица 3.1 – Проверки правдоподобия User-Agent (правило 3)

Код	Проверка
3a	версия браузера выше существующей (с запасом на beta/canary)
3b	номер сборки Chrome не соответствует мажорной версии
3c	несуществующая версия Windows NT
3d	версия браузера не поддерживает заявленную Windows (7/8, XP)
3e	версии Edg/ и Chrome/ не совпадают
3f	rv: не совпадает с версией Firefox
3g	современный Safari на Windows
3h	полная сборка в Chrome, который обязан урезать User-Agent
3i	Chrome на Android без урезанного «Android 10; K» (слабая проверка)
3j	версия Chrome не поддерживает заявленный Android
3k	невозможная версия macOS для браузера
3l	средство автоматизации (curl, python-requests, HeadlessChrome и др.) или пустой User-Agent
3m	версия Safari не соответствует версии iOS

При переносе прототипа в модуль обнаружена ошибка: проверки 3h–3m стояли после оператора return и никогда не выполнялись, а также ссылались на неопределённые константы. Ошибка исправлена, недостающие таблицы версий добавлены. Прогон на реальном журнале выявил ложные срабатывания: проверка 3h реагировала на Chrome для iOS и приложения Яндекса, которые не урезают User-Agent, – добавлены исключения; проверка 3i дала 229 ложных срабатываний на встроенных браузерах приложений, поэтому она помечена как слабая и сама по себе правило не включает.

3.3 Эвристические правила

Правила реализованы по шаблону «Стратегия» [27]: каждое правило – класс с общим интерфейсом Rule, у которого два метода – prepare (глобальная подготовка по всему журналу) и check (проверка одного IP, возвращает подробности или None). Движок RuleEngine сначала вызывает prepare у всех правил по возрастанию номера, затем check для каждого IP (листинг А.2). Новое правило добавляется без изменения движка.

– **Правило 1 – поддельный поисковый робот.** Класс DnsVerifier выполняет обратный DNS-запрос, проверяет домен (googlebot.com, google.com, yandex.ru, yandex.net, yandex.com, search.msn.com и др.) и прямым запросом убеждается, что имя разрешается в тот же IP. Запросы выполняются в 50 потоков, результаты кэшируются в файле, IP без ответа DNS получают статус «неизвестно» и поддельными не считаются. Функции запросов передаются в конструктор, что позволяет подменять DNS в тестах. Подтверждённые роботы исключаются из всех правил;
– **Правило 2 – разные браузеры/ОС на одном IP** (слабый сигнал);
– **Правило 3 – невозможный User-Agent** (раздел 3.2);
– **Правило 5 – аномально равномерные интервалы.** Запросы с разрывом до 1 с склеиваются в один визит (загрузка страницы со статикой), по интервалам между началами визитов считается коэффициент вариации $CV = \sigma / \mu$; при $CV < 0{,}15$ и не менее пяти визитах правило срабатывает;
– **Правило 6 – много IP одной подсети в коротком окне.** Для каждой подсети /24 (/64 для IPv6) скользящим окном длиной 600 с подсчитывается число разных активных IP; помечаются только IP, активные в окне, где их число достигло пяти, а не вся подсеть.

Правило 4 прототипа было удалено ранее, нумерация сохранена для совместимости отчётов.

3.4 Реализация ML-компонента

Генерация разметки и подготовка данных

Класс BotSimulator собирает из журнала HTML-страницы с реальными размерами ответов, статические файлы и User-Agent, встречающиеся хотя бы у трёх IP, и генерирует ботов четырёх типов (таблица 2.4); функция inject_bots возвращает новый журнал и разметку. Функция extract_features строит таблицу признаков pandas, build_dataset добавляет метки, флаг срабатывания правил и

выполняет стратифицированное разбиение (листинг А.3). На маленьких журналах редкие группы объединяются, а при необходимости разбиение выполняется без стратификации.

Модели

Все модели реализуют интерфейс AnomalyModel: fit(X, X_val), score(X) – чем больше, тем аномальнее, feature_contributions(X) для объяснения, get_state() и from_state() для сохранения. Фабрика create_model создаёт модель по имени из реестра MODEL_REGISTRY. Класс AnomalyDetector объединяет модель со StandardScaler, порогом и списком признаков и предоставляет методы fit, score, predict, explain, save и load.

Автоэнкодер (листинг А.1) реализован на PyTorch. Архитектура 21 → 16 → 8 → 3 → 8 → 16 → 21 с функцией активации ReLU выбрана перебором 18 вариантов (раздел 4.2). Обучение: оптимизатор Adam (скорость 0,001, weight_decay 0,00001), функция потерь MSE, размер пакета 128, до 300 эпох, ранняя остановка, если ошибка на отложенных данных не улучшается 25 эпох, – после неё восстанавливаются лучшие веса. Оценка аномальности – средний квадрат ошибки восстановления, а ошибка по каждому признаку используется для объяснения: три признака с наибольшей ошибкой выводятся в отчёт. Модель сохраняется как state_dict вместе с параметрами архитектуры и историей обучения.

Isolation Forest – 300 деревьев, оценка – score_samples со знаком минус. **One-Class SVM** (nu = 0,05, RBF-ядро) и **Local Outlier Factor** (20 соседей, режим novelty) реализованы для сравнения и в работе детектора по умолчанию не используются.

Подбор порога

Порог подбирается на валидационной выборке по максимуму F1 с помощью кривой точности-полноты. При отладке веб-интерфейса обнаружена ошибка: функция precision_recall_curve считает положительными оценки «не меньше порога», а детектор сравнивал «строго больше», поэтому IP, на оценке

которого стоял порог, выпадал из ботов. Теперь порог ставится посередине между оценкой граничного IP и ближайшей меньшей оценкой; ошибка закреплена регрессионным тестом.

Итоговое решение

Класс DetectionService (листинг А.4) – фасад системы: применяет правила, извлекает признаки, получает оценки всех загруженных моделей и формирует для каждого IP вердикт IPVerdict с причиной «ruleN» или «model:autoencoder». Модель-«решатель» по умолчанию – автоэнкодер, остальные модели только добавляют оценки в отчёт. Модуль report выгружает результат в TXT (одна строка на IP с человекочитаемыми причинами) и JSON. Командная строка (python -m lsdetector) предоставляет команды detect, train, simulate, demo и sample.

3.5 Реализация бэкенда

REST API реализован на FastAPI [25] (таблица 3.2). Параметры анализа описаны Pydantic-моделью AnalyzeParams с ограничениями значений (например, порог CV в интервале (0; 1)), ответы – моделями AnalysisResponse, IPResult, Summary и др. Модели и соединение с базой создаются один раз на процесс.

Таблица 3.2 – Эндпоинты REST API

Метод	Путь	Назначение
GET	/api/health	состояние сервиса, версия, загруженные модели
GET	/api/models	пороги, признаки и метрики моделей
POST	/api/analyze	загрузить журнал (multipart), параметры – в строке запроса
GET	/api/analyses	история анализов, новые первыми
GET	/api/analyses/{id}	результат анализа (only_bots=true – только боты)
DELETE	/api/analyses/{id}	удалить анализ вместе с решениями по IP
GET	/api/analyses/{id}/download	JSON с ботами для выгрузки
GET	/api/ips/{ip}	решения по одному IP во всех анализах

Хранилище AnalysisRepository – слой доступа к данным поверх sqlite3 (шаблон «Репозиторий»): методы save, get, list, ip_history и delete. Схема создаётся из файла schema.sql при первом подключении; анализ и все его IP записываются в одной транзакции, удаление каскадное. Обработчики FastAPI выполняются в пуле потоков, поэтому соединение защищено блокировкой; режим WAL и тайм-аут ожидания 30 с позволяют запускать несколько процессов uvicorn. Последовательность обработки запроса на анализ показана на рисунке 3.1.

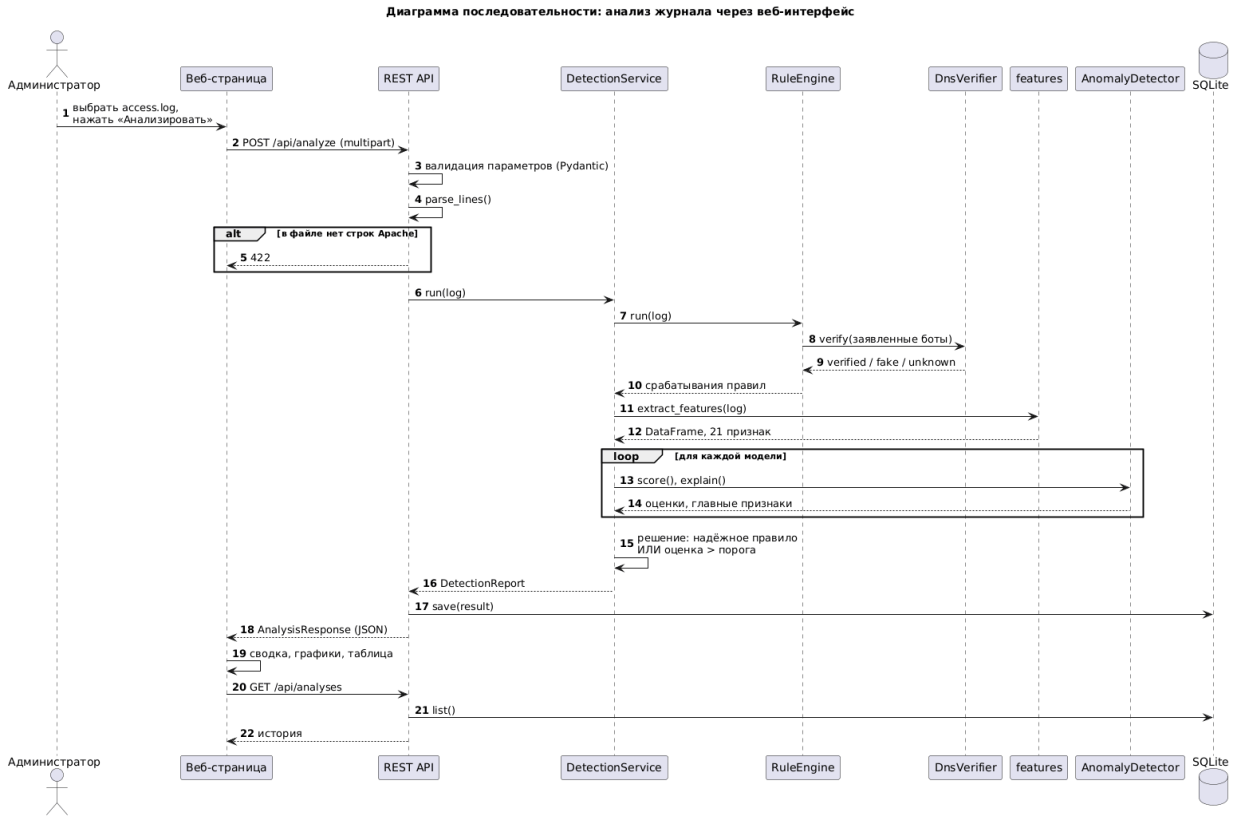


Рисунок 3.1 – Диаграмма последовательности анализа журнала

Нагрузочный тест (раздел 4.5) выявил, что блокировка файла кэша DNS охватывала весь анализ, и запросы выполнялись строго по очереди. Блокировка сужена до чтения и записи файла: перед сохранением кэш перечитывается, и новые записи дописываются к результатам других запросов.

3.6 Реализация фронтенда

Веб-страница написана на HTML, CSS и JavaScript без фреймворков и внешних библиотек. Данные запрашиваются функцией fetch: загрузка файла отправляется как multipart/form-data на /api/analyze, история и сохранённые результаты читаются из соответствующих эндпоинтов. Графики строятся в виде SVG: горизонтальная диаграмма причин и гистограмма оценок в логарифмической шкале с отметкой порога. Таблица IP фильтруется, сортируется по любому столбцу и показывает до 500 строк; текст из журнала экранируется перед вставкой в HTML. Цвета заданы CSS-переменными с тёмной темой по prefers-color-scheme.

3.7 Принципы качественного кода

Код следует принципам SOLID [28, 29]:

- **единственная ответственность (SRP):** каждый модуль отвечает за одно – разбор, правила, признаки, модели, оценку, вывод, хранение; DetectionService только оркестрирует;
- **открытость/закрытость (OCP):** новое правило – новый класс Rule, новая модель – новый класс AnomalyModel в реестре; движок и сервис не меняются;
- **подстановка Лисков (LSP):** четыре модели взаимозаменяемы в AnomalyDetector, одни и те же тесты прогоняются для каждой;
- **разделение интерфейсов (ISP):** интерфейс модели содержит четыре обязательных метода, объяснение – необязательный метод с реализацией по умолчанию;
- **инверсия зависимостей (DIP):** DnsVerifier получает функции DNS-запросов извне, DetectionService – готовые verifier и модели.

Использованы шаблоны проектирования: Стратегия (правила), Фабрика (create_model), Фасад (DetectionService), Репозиторий (AnalysisRepository), Одиночка для загруженных моделей и соединения с БД в API (lru_cache). Стиль

кода соответствует PEP 8 и проверяется ruff в CI; имена функций и переменных осмысленны, комментарии поясняют причины решений, а не пересказывают код. Диаграмма основных классов приведена в приложении Б.

Выводы по главе 3

Реализованы все компоненты системы: разбор журнала и анализ User-Agent с исправлением ошибок прототипа, пять эвристических правил по шаблону «Стратегия», извлечение 21 признака, генератор синтетических ботов, четыре модели поиска аномалий, включая нейросетевой автоэнкодер на PyTorch, подбор порога, итоговое решение с объяснением, REST API на FastAPI с хранилищем SQLite и веб-интерфейс. Разработка велась в Git по стратегии main – develop – feature с непрерывной интеграцией. Код структурирован по принципам SOLID и DRY, оформлен по PEP 8 и готов к тестированию.

4 ТЕСТИРОВАНИЕ И ОЦЕНКА РЕЗУЛЬТАТОВ

4.1 Тест-план и автоматические тесты

Тест-план охватывает модульное, интеграционное, функциональное и нагрузочное тестирование (таблица 4.1). Автоматические тесты используют только синтетический журнал и поддельный DNS, поэтому выполняются без сети и реальных данных; 64 теста проходят примерно за 11 с и запускаются в GitHub Actions на каждый push.

Таблица 4.1 – Тест-план

Уровень	Что проверяется	Средство
Модульный	разбор строк, время, IPv6, статистика пропусков	test_parser.py
Модульный	13 проверок User-Agent, исключения для роботов и браузеров	test_useragent.py
Модульный	правила 1–6 с поддельным DNS, белый список, приоритет правила 1	test_rules.py
Модульный	признаки, выборки, порог, обучение и сохранение четырёх моделей, отчёт оценки	test_ml.py
Модульный	хранилище: сохранение, история, каскадное удаление	test_storage.py
Интеграционный	DetectionService, CLI, REST API (успех и ошибки), история, веб-страница	test_pipeline_api.py
Нагрузочный	параллельные запросы к API	load_test.py
Производительность	полный журнал 395 тыс. строк	ручной прогон

4.2 Тестирование ML-компонента

Данные. Реальный журнал (2 254 IP с пятью и более запросами) и 516 синтетических ботов; всего 2 770 IP. Тестовая выборка: 104 бота и 326 реальных IP, на которых не сработало ни одно надёжное правило. Поэтому точность «только правил» на ней равна единице по построению, а реальные IP, найденные правилами, оцениваются отдельно.

Сравнение моделей. Чтобы выбор не зависел от одного удачного разбиения, сравнение повторено на пяти вариантах внедрения ботов (таблица

4.2); для каждого заново строятся выборки, обучаются модели и подбирается порог.

Таблица 4.2 – Сравнение моделей (5 вариантов данных, среднее ± стандартное отклонение)

Модель	Precision	Recall	F1	ROC-AUC	Обучение, с	Оценка 1000 IP, мс	Размер, КБ
Isolation Forest	0,838	0,981	0,904 ± 0,013	0,965 ± 0,007	0,35	63,0	3 407
Автоэнкодер	0,850	1,000	0,919 ± 0,017	0,969 ± 0,010	3,98	2,5	15
One-Class SVM	0,800	0,959	0,872 ± 0,012	0,969 ± 0,004	0,01	7,0	16
LOF	0,813	0,996	0,895 ± 0,022	0,950 ± 0,016	0,04	26,1	331

Автоэнкодер показал лучший средний F1 и полноту, по ROC-AUC он делит первое место с One-Class SVM. Отрыв от Isolation Forest по F1 сопоставим с разбросом между вариантами данных, поэтому учитывались и практические свойства: оценка в 25 раз быстрее Isolation Forest, модель занимает 15 КБ вместо 3,4 МБ, а ошибка по признакам объясняет решение. Обучение дольше (около 4 с), но выполняется редко. Автоэнкодер выбран основной моделью.

Подбор гиперпараметров. Перебраны 18 архитектур автоэнкодера: скрытые слои 16-8, 32-16, 64-32, размер латентного слоя 3, 6, 10, dropout 0 и 0,1. Выбор выполнялся по F1 на валидационной выборке (таблица 4.3).

Таблица 4.3 – Подбор архитектуры автоэнкодера (лучшие и худший варианты)

Скрытые слои	Латентный слой	Dropout	F1 (валидация)	Эпох	F1 (тест)	ROC-AUC (тест)
16-8	3	0	0,928	300	0,963	0,990
16-8	3	0,1	0,924	184	0,972	0,989
16-8	6	0	0,920	183	0,954	0,989
32-16	10	0,1	0,920	288	0,959	0,983
64-32	3	0	0,833	255	0,869	0,962

Узкая сеть работает лучше широкой: широкая успевает научиться восстанавливать и необычные векторы, и ошибка ботов перестаёт отличаться от ошибки людей. Первоначальная архитектура 32-16-6 (F1 на валидации 0,904) заменена на 16-8-3.

Итоговые метрики. Результаты основных методов на тестовой выборке приведены в таблице 4.4, полнота по типам ботов – в таблице 4.5, матрицы ошибок – на рисунке 4.1, ROC- и PR-кривые – на рисунке В.1 приложения В.

Таблица 4.4 – Метрики на тестовой выборке

Метод	Precision	Recall	F1	ROC-AUC
Только правила	1,000	0,529	0,692	–
Isolation Forest	0,866	0,990	0,924	0,970
Правила + Isolation Forest	0,867	1,000	0,929	–
Автоэнкодер (нейросеть)	0,929	1,000	0,963	0,990
Правила + автоэнкодер	0,929	1,000	0,963	–

Таблица 4.5 – Полнота по типам ботов

Тип бота	Правила	Isolation Forest	Автоэнкодер	Правила + автоэнкодер
timer	0,55	1,00	1,00	1,00
random_slow	0,00	1,00	1,00	1,00
distributed	1,00	0,98	1,00	1,00
mimic	0,00	1,00	1,00	1,00



Рисунок 4.1 – Матрицы ошибок на тестовой выборке

Целевые метрики НФТ-2 (F1 ≥ 0,85, Precision ≥ 0,9) достигнуты комбинацией «правила + автоэнкодер». На пяти вариантах данных средняя точность автоэнкодера ниже (0,85), поэтому требование к точности выполняется не на каждом варианте – это учитывается в анализе ошибок. Правила ловят только ботов с явным признаком (таймер – правило 5, группа в подсети – правило 6) и полностью пропускают ботов со случайными интервалами и подгрузкой статики; модель находит их по поведению.

Распределение оценок (рисунок В.2 приложения В) показывает, что ошибки восстановления людей и ботов различаются на порядок. По средней ошибке восстановления (рисунок В.3) видно, по каким признакам боты сильнее всего отличаются от людей: доля запросов без Referer, доля статики, медианный интервал и энтропия интервалов.

Согласованность с правилами. Из 124 реальных IP тестовой выборки, найденных правилами (в основном поддельные Googlebot и YandexBot), автоэнкодер сам считает аномальными 96,0 %, Isolation Forest – 97,6 %, хотя эти IP в обучении не участвовали. Это подтверждает, что модель выделяет реальных ботов, а не только синтетических.

4.3 Анализ ошибок

- **Пропуски.** Автоэнкодер на тестовой выборке ботов не пропустил. Isolation Forest пропустил одного бота типа distributed с несколькими запросами – по признакам он похож на случайного посетителя; его находит правило 6;
- **Ложные срабатывания автоэнкодера** – 8 из 326 реальных IP (2,5 %). Чаще всего среди главных признаков – доля запросов с параметрами, доля уникальных путей, доля запросов без Referer, число разных User-Agent: это посетители, активно пользующиеся поиском и фильтрами, приходящие по прямым ссылкам или находящиеся за общим NAT. IP с наибольшей оценкой выделяется долей ответов 5xx – это скорее сбой на стороне сервера. Часть таких IP может оказаться настоящими ботами, которых не ловят правила; поэтому в отчёте для каждого IP указаны главные признаки, а решение о блокировке остаётся за администратором;
- **Ограничения оценки.** Синтетические боты построены по нашему же представлению о low-and-slow поведении, поэтому метрики на них оптимистичны. Модели нужно обучать под конкретный сайт: модели, обученные на журнале магазина, на демонстрационном журнале

вымышленного сайта давали сотни ложных срабатываний, а после обучения на нём – 12 из 315 реальных IP при найденных 59 из 59 ботов.

4.4 Функциональное тестирование

API тестировалось автоматически (таблица 4.6) и вручную через Swagger (рисунок В.4 приложения В). Пример запроса:

```
curl -F "file=@data/demo/demo_access.log" \
  "http://127.0.0.1:8000/api/analyze?dns_offline=true"
```

В ответе для демонстрационного журнала: 20 773 строки, 374 IP, 71 подозрительный IP (39 найдены моделью, 23 – правилом 6, 9 – правилом 5); для каждого IP – правила, оценки моделей и описание, например «аномально равномерные интервалы (CV = 0,0255), аномалия autoencoder = 5,647 (признаки: interval_entropy, log_median_interval, empty_referer_share)».

Таблица 4.6 – Сценарии функционального тестирования API

Сценарий	Ожидаемый результат	Тест
Загрузка журнала	200, сводка, IP с описаниями	test_api_analyze_flow
Повторное открытие результата	тот же список IP и модель	test_api_analyze_flow
История анализов и история IP	новый анализ первым, IP найден	test_api_analyze_flow
Удаление анализа	204, затем 404; история IP пуста	test_api_analyze_flow
Файл не в формате Apache	422	test_api_errors
Недопустимый параметр (cv_threshold = 5)	422 с описанием ошибки	test_api_errors
Неизвестный анализ	404	test_api_errors

Веб-интерфейс проверен вручную (рисунок 4.2): загрузка файла, сводка, графики причин и оценок с порогом, таблица с поиском, фильтрами и сортировкой, выгрузка JSON, история анализов (открыть, удалить), открытие результата по ссылке. Все функциональные требования ФТ-1…ФТ-16 покрыты автоматическими тестами или ручной проверкой; соответствие требований и тестов приведено в документации проекта (docs/04_testing.md).

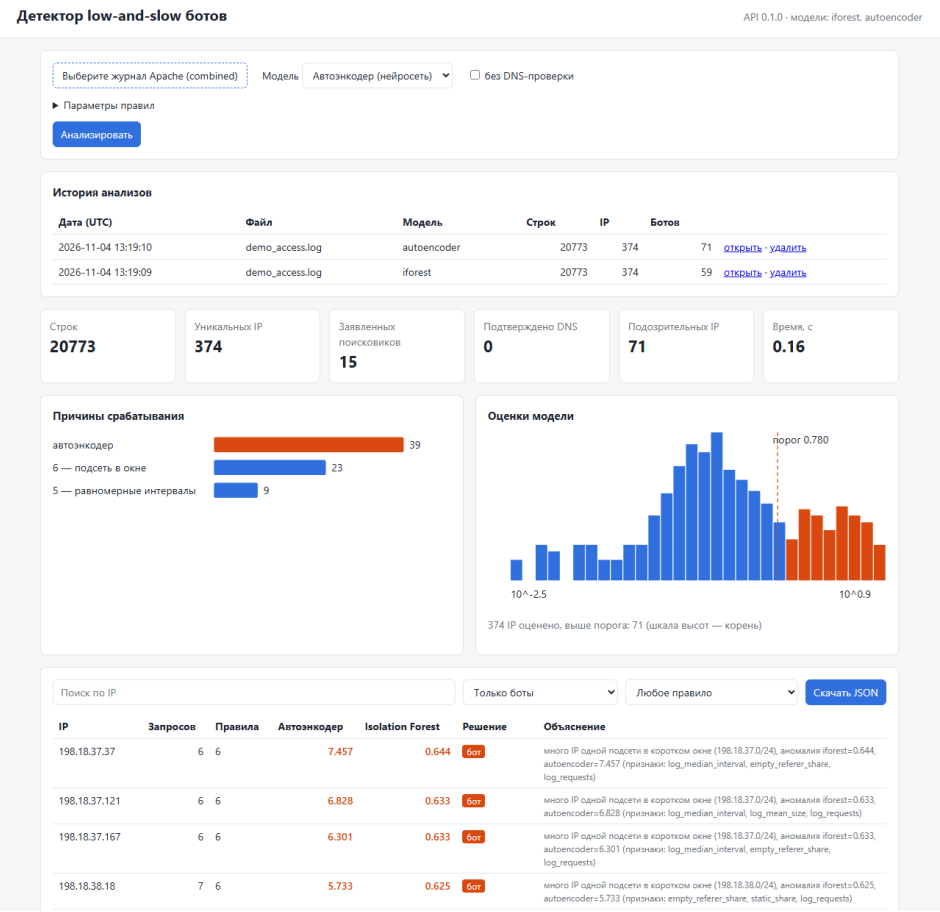


Рисунок 4.2 – Веб-интерфейс: результат анализа демонстрационного журнала

4.5 Производительность и нагрузочное тестирование

Полный журнал (395 тыс. строк, 150 МБ) с правилами и обеими моделями при кэшированных DNS-ответах обрабатывается за 19 с на ноутбуке с 8 логическими ядрами – требование НФТ-1 (не более 60 с) выполнено. Первый прогон с DNS-проверкой 2 955 заявленных роботов занимает около 3 минут.

Нагрузочный тест API выполнялся скриптом load_test.py: 24 запроса POST /api/analyze с демонстрационным журналом (20 773 строки, 4,9 МБ) на каждом уровне параллельности (таблица 4.7).

Таблица 4.7 – Результаты нагрузочного тестирования

Процессов uvicorn	Параллельных запросов	Успешно	Запросов в секунду	p50, с	p95, с
1	1	24 / 24	1,26	0,77	0,86
1	4	24 / 24	1,35	2,95	3,65
1	8	24 / 24	1,33	5,90	8,97
4	1	24 / 24	1,37	0,72	0,79
4	4	24 / 24	3,06	0,73	1,50
4	8	24 / 24	3,13	1,88	6,09

В одном процессе пропускная способность не растёт с числом потоков: разбор журнала и расчёт признаков выполняются кодом на Python, и глобальная блокировка интерпретатора (GIL) не позволяет им идти параллельно. Масштабирование достигается несколькими процессами: с четырьмя процессами пропускная способность выросла в 2,3 раза. При первом прогоне с несколькими процессами возникала ошибка «database is locked»; она устранена переводом SQLite в режим WAL с тайм-аутом ожидания.

4.6 Найденные и исправленные ошибки

В ходе разработки и тестирования найдено и исправлено восемь дефектов (таблица В.1 приложения В): три – в проверках User-Agent прототипа, два – в подборе порога и отображении результатов, один – в разбиении выборок на маленьком журнале, два – в параллельной обработке запросов API, выявленные нагрузочным тестом. Для исправленных ошибок в логике добавлены регрессионные тесты.

4.7 Оценка эффективности

Сравнение исходного прототипа и разработанной системы приведено в таблице 4.8.

Таблица 4.8 – Сравнение AS-IS / TO-BE

Показатель	**AS-IS (прототип)**	**TO-BE (система)**
Методы	5 правил	правила + нейросетевой автоэнкодер (+ 3 модели для сравнения)
Полнота на low-and-slow ботах	0,53	1,00
Точность	1,00	0,93
Объяснение решения	номер правила	правило + главные признаки модели
Интерфейсы	командная строка	командная строка, REST API, веб-страница
Хранение результатов	файл	SQLite: история анализов и IP
Тесты	нет	64 теста, непрерывная интеграция

Для администратора это означает: из каждых 100 медленных ботов прототип находил около 53, а система – практически всех, ценой 2,5 % ложных срабатываний на обычных IP, каждое из которых сопровождается объяснением для ручной проверки. На реальном журнале система нашла 2 516 подозрительных IP, из них 49 – только моделью, без срабатывания какого-либо правила.

Выводы по главе 4

Разработан тест-план, написаны 64 автоматических теста, проведены функциональное, нагрузочное тестирование и проверка производительности. Сравнение четырёх моделей на пяти вариантах данных и перебор 18 архитектур обосновали выбор автоэнкодера 16-8-3. Комбинация правил и автоэнкодера достигла на тестовой выборке Precision 0,929, Recall 1,000, F1 0,963, что выполняет целевые требования. Проведён анализ ошибок и ограничений, исправлены восемь найденных дефектов. Полнота обнаружения по сравнению с прототипом выросла с 0,53 до 1,00, что подтверждает практическую ценность решения.

ЗАКЛЮЧЕНИЕ

В ходе курсового проекта разработана программная система обнаружения low-and-slow ботов по журналам веб-сервера Apache. Все поставленные задачи выполнены:

1) проанализирована предметная область и существующие решения, показана массовая подделка поисковых роботов на реальном журнале, сформулированы 16 функциональных и 7 нефункциональных требований и техническое задание
2) спроектирована архитектура модульного монолита (диаграммы C4 и UML), модель данных с хранением истории в SQLite и ML-пайплайн с разметкой внедрением синтетических ботов, обоснован выбор технологий
3) реализованы разбор журнала, пять эвристических правил, 21 поведенческий признак, четыре модели поиска аномалий, в том числе нейросетевой автоэнкодер на PyTorch, REST API на FastAPI и веб-интерфейс; исправлены ошибки исходного прототипа
4) проведено тестирование (64 автоматических теста, функциональное и нагрузочное тестирование) и оценка качества: комбинация правил и автоэнкодера достигла F1 0,963 при точности 0,929

Цель проекта достигнута: система находит медленных ботов, которых не замечают правила, объясняет каждое решение и обрабатывает журнал из 400 тыс. строк менее чем за 20 с. Исходный код, документация, обученные модели и демонстрационные данные опубликованы в открытом репозитории.

Практическая ценность. Система может использоваться администраторами интернет-магазинов и других сайтов для регулярного анализа журналов и передачи списка IP фаерволу или системе мониторинга через REST API, не передавая данные посетителей третьим лицам.

Пути дальнейшего развития:

– ручная разметка выборки реальных IP для оценки качества без синтетических данных и активное обучение по отметкам администратора;
– потоковый режим анализа журнала в реальном времени и прямая интеграция с fail2ban, ipset или облачными фаерволами;
– поддержка других форматов журналов (nginx, IIS) и дополнительных признаков – автономная система и страна IP, последовательность страниц;
– переход на PostgreSQL и контейнеризация (Docker) для многопользовательского развёртывания.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

Оформление записки выполнено с учётом ГОСТ 7.32-2017 [30].

1. Imperva. 2024 Bad Bot Report [Электронный ресурс]. – URL: https://www.imperva.com/resources/resource-library/reports/2024-bad-bot-report/ (дата обращения: 04.11.2026).

2. OWASP Automated Threats to Web Applications [Электронный ресурс]. – URL: https://owasp.org/www-project-automated-threats-to-web-applications/ (дата обращения: 04.11.2026).

3. Apache HTTP Server Version 2.4 Documentation. Module mod_log_config [Электронный ресурс]. – URL: https://httpd.apache.org/docs/2.4/mod/mod_log_config.html (дата обращения: 04.11.2026).

4. Google Search Central. Verifying Googlebot and other Google crawlers [Электронный ресурс]. – URL: https://developers.google.com/search/docs/crawling-indexing/verifying-googlebot (дата обращения: 04.11.2026).

5. Яндекс Вебмастер. Как проверить, что робот принадлежит Яндексу [Электронный ресурс]. – URL: https://yandex.ru/support/webmaster/robot-workings/check-yandex-robots.html (дата обращения: 04.11.2026).

6. Fail2Ban [Электронный ресурс] : репозиторий проекта. – URL: https://github.com/fail2ban/fail2ban (дата обращения: 04.11.2026).

7. CrowdSec Documentation [Электронный ресурс]. – URL: https://docs.crowdsec.net/ (дата обращения: 04.11.2026).

8. Cloudflare Bot Management Documentation [Электронный ресурс]. – URL: https://developers.cloudflare.com/bots/ (дата обращения: 04.11.2026).

9. GoAccess – Visual Web Log Analyzer [Электронный ресурс]. – URL: https://goaccess.io/ (дата обращения: 04.11.2026).

10. ГОСТ 34.602-89. Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. – М. : Стандартинформ, 2009.

11. ГОСТ Р ИСО/МЭК 12207-2010. Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств. – М. : Стандартинформ, 2011.

12. Brown S. The C4 model for visualising software architecture [Электронный ресурс]. – URL: https://c4model.com/ (дата обращения: 04.11.2026).

13. SQLite Documentation. Write-Ahead Logging [Электронный ресурс]. – URL: https://www.sqlite.org/wal.html (дата обращения: 04.11.2026).

14. Chandola V., Banerjee A., Kumar V. Anomaly detection: A survey // ACM Computing Surveys. – 2009. – Vol. 41, № 3. – Article 15.

15. Liu F. T., Ting K. M., Zhou Z.-H. Isolation Forest // Proceedings of the 8th IEEE International Conference on Data Mining (ICDM). – 2008. – P. 413–422.

16. Schölkopf B., Platt J. C., Shawe-Taylor J., Smola A. J., Williamson R. C. Estimating the support of a high-dimensional distribution // Neural Computation. – 2001. – Vol. 13, № 7. – P. 1443–1471.

17. Breunig M. M., Kriegel H.-P., Ng R. T., Sander J. LOF: identifying density-based local outliers // Proceedings of the ACM SIGMOD International Conference on Management of Data. – 2000. – P. 93–104.

18. Hinton G. E., Salakhutdinov R. R. Reducing the dimensionality of data with neural networks // Science. – 2006. – Vol. 313, № 5786. – P. 504–507.

19. Гудфеллоу Я., Бенджио И., Курвилль А. Глубокое обучение. – М. : ДМК Пресс, 2018.

20. Маккини У. Python и анализ данных. – М. : ДМК Пресс, 2020.

21. Scikit-learn: Machine Learning in Python. Documentation [Электронный ресурс]. – URL: https://scikit-learn.org/stable/ (дата обращения: 04.11.2026).

22. Рашка С. Python и машинное обучение. – М. : ДМК Пресс, 2019.

23. PyTorch Documentation [Электронный ресурс]. – URL: https://pytorch.org/docs/stable/ (дата обращения: 04.11.2026).

24. Шолле Ф. Глубокое обучение на Python. – СПб. : Питер, 2018.

25. FastAPI Documentation [Электронный ресурс]. – URL: https://fastapi.tiangolo.com/ (дата обращения: 04.11.2026).

26. The Chromium Projects. User-Agent Reduction [Электронный ресурс]. – URL: https://www.chromium.org/updates/ua-reduction/ (дата обращения: 04.11.2026).

27. Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Паттерны объектно-ориентированного проектирования. – СПб. : Питер, 2020.

28. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения. – СПб. : Питер, 2020.

29. Мартин Р. Чистый код. Создание, анализ и рефакторинг. – СПб. : Питер, 2019.

30. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. – М. : Стандартинформ, 2017.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинги программного кода

Полные исходные тексты – в репозитории https://github.com/Gregory7635/low-and-slow-detector.

Листинг А.1 – Нейросетевой автоэнкодер (lsdetector/models/autoencoder.py)

```
"""Нейросетевой автоэнкодер (PyTorch) для поиска аномального поведения IP.

Сеть сжимает вектор признаков до узкого скрытого слоя и восстанавливает
его обратно. Обучается только на обычных посетителях, поэтому поведение
ботов восстанавливает хуже: ошибка восстановления (MSE) и есть оценка
аномальности. Ошибка по каждому признаку показывает, чем именно IP
отличается от нормы.
"""

import copy

import numpy as np
import torch
from torch import nn

from .base import AnomalyModel



class _AutoencoderNet(nn.Module):
    def __init__(self, n_features, hidden=(16, 8), latent=3, dropout=0.0):
        super().__init__()
        enc, prev = [], n_features
        for h in hidden:
            enc += [nn.Linear(prev, h), nn.ReLU(), nn.Dropout(dropout)]
            prev = h
        enc.append(nn.Linear(prev, latent))
        dec, prev = [], latent
        for h in reversed(hidden):
            dec += [nn.Linear(prev, h), nn.ReLU()]
            prev = h
        dec.append(nn.Linear(prev, n_features))
        self.encoder = nn.Sequential(*enc)
        self.decoder = nn.Sequential(*dec)

    def forward(self, x):
        return self.decoder(self.encoder(x))



class AutoencoderModel(AnomalyModel):
    name = "autoencoder"

    # Архитектура выбрана по F1 на validation (scripts/experiments.py)
    def __init__(self, hidden=(16, 8), latent=3, dropout=0.0, lr=1e-3,
                 weight_decay=1e-5, epochs=300, batch_size=128, patience=25,
```

```
                 random_state=42, device=None):
        self.params = dict(hidden=tuple(hidden), latent=latent, dropout=dropout,
lr=lr,
                           weight_decay=weight_decay, epochs=epochs,
batch_size=batch_size,
                           patience=patience, random_state=random_state)
        # Сеть маленькая, на CPU она обучается быстрее, чем с копированием на
GPU
        self.device = torch.device(device or "cpu")
        self.net = None
        self.n_features = None
        self.history = {"train_loss": [], "val_loss": []}

    def _tensor(self, X):
        return torch.as_tensor(np.asarray(X, dtype=np.float32),
device=self.device)

    def _build(self, n_features):
        p = self.params
        self.n_features = n_features
        self.net = _AutoencoderNet(n_features, p["hidden"], p["latent"],
                                   p["dropout"]).to(self.device)

    def fit(self, X, X_val=None):
        p = self.params
        torch.manual_seed(p["random_state"])
        rng = np.random.default_rng(p["random_state"])
        X = np.asarray(X, dtype=np.float32)
        if X_val is None:   # отложить 15 % обучающих данных для ранней
остановки
            idx = rng.permutation(len(X))
            cut = max(1, int(0.15 * len(X)))
            X, X_val = X[idx[cut:]], X[idx[:cut]]
        self._build(X.shape[1])
        opt = torch.optim.Adam(self.net.parameters(), lr=p["lr"],
                               weight_decay=p["weight_decay"])
        loss_fn = nn.MSELoss()
        xt, xv = self._tensor(X), self._tensor(X_val)
        best, best_state, wait = float("inf"), None, 0
        for _ in range(p["epochs"]):
            self.net.train()
            perm = torch.randperm(len(xt))
            total = 0.0
            for i in range(0, len(xt), p["batch_size"]):
                batch = xt[perm[i:i + p["batch_size"]]]
                opt.zero_grad()
                loss = loss_fn(self.net(batch), batch)
                loss.backward()
                opt.step()
                total += loss.item() * len(batch)
            self.net.eval()
            with torch.no_grad():
                val = loss_fn(self.net(xv), xv).item()
            self.history["train_loss"].append(total / len(xt))
            self.history["val_loss"].append(val)
            if val < best - 1e-5:
                best, best_state, wait = val,
copy.deepcopy(self.net.state_dict()), 0
            else:
                wait += 1
                if wait >= p["patience"]:
                    break
        self.net.load_state_dict(best_state)
```

```
        self.net.eval()
        return self

    def reconstruction_errors(self, X):
        """Квадрат ошибки восстановления по каждому признаку."""
        with torch.no_grad():
            x = self._tensor(X)
            return ((self.net(x) - x) ** 2).cpu().numpy()

    def score(self, X):
        return self.reconstruction_errors(X).mean(axis=1)

    def feature_contributions(self, X):
        return self.reconstruction_errors(X)

    def get_state(self):
        return {"params": self.params, "n_features": self.n_features,
                "state_dict": {k: v.cpu() for k, v in
self.net.state_dict().items()},
                "history": self.history}

    @classmethod
    def from_state(cls, state):
        obj = cls(**state["params"])
        obj._build(state["n_features"])
        obj.net.load_state_dict(state["state_dict"])
        obj.net.eval()
        obj.history = state.get("history", obj.history)
        return obj
```

Листинг А.2 – Интерфейс правила, правило 5 и движок правил (фрагменты lsdetector/rules.py)

```
class Rule(ABC):
    id: int
    name: str

    def prepare(self, ctx):
        """Глобальная подготовка по всему журналу (по умолчанию не нужна)."""

    @abstractmethod
    def check(self, ip, records, ctx):
        """Вернуть словарь с подробностями, если правило сработало, иначе
None."""




...

def visit_starts(times, burst_gap):
    """Склеить запросы с разрывом <= burst_gap в один визит."""
    starts, prev = [], None
    for t in times:
        if prev is None or t - prev > burst_gap:
            starts.append(t)
        prev = t
    return starts
```

```
def interval_cv(times, burst_gap, min_visits):
    """Коэффициент вариации интервалов между визитами или None (мало данных)."""
    starts = visit_starts(times, burst_gap)
    if len(starts) < min_visits:
        return None
    iv = [b - a for a, b in zip(starts, starts[1:])]
    avg = mean(iv)
    return pstdev(iv) / avg if avg > 0 else None



class RegularIntervalsRule(Rule):
    id, name = 5, RULE_NAMES[5]

    def check(self, ip, records, ctx):
        c = ctx.config
        cv = interval_cv([r.ts for r in records], c.rule5_burst_gap,
c.rule5_min_visits)
        if cv is not None and cv < c.rule5_cv_threshold:
            return {"rule5_cv": round(cv, 4)}
        return None



...

class RuleEngine:
    def __init__(self, rules, config):
        self.rules = rules
        self.config = config

    def run(self, log):
        ctx = RuleContext(self.config, log, whitelist=[
            ipaddress.ip_network(n, strict=False) for n in
self.config.whitelist])
        # правило 1 готовится первым: от его результата зависят остальные
        for rule in sorted(self.rules, key=lambda r: r.id):
            rule.prepare(ctx)

        hits = {}
        for ip, records in log.by_ip.items():
            if ip in ctx.verified:
                continue
            rules, details = set(), {}
            for rule in sorted(self.rules, key=lambda r: r.id):
                det = rule.check(ip, records, ctx)
                if det is None:
                    continue
                rules.add(rule.id)
                details.update(det)
                if rule.id == 1 and self.config.rule1_priority:
                    break
            if rules:
                hits[ip] = {"rules": rules, "details": details}
        return RuleResults(hits, ctx.verified, ctx.unknown, len(ctx.dns),
ctx.rule6_report)
```

Листинг А.3 – Подготовка выборок, обучение и подбор порога (lsdetector/training.py)

```
"""Подготовка выборок, обучение моделей и подбор порога."""

import time
from dataclasses import dataclass

import numpy as np
import pandas as pd
from sklearn.metrics import precision_recall_curve
from sklearn.model_selection import train_test_split

from .features import FEATURE_NAMES, extract_features
from .models import AnomalyDetector, create_model
from .rules import STRONG_RULES



@dataclass
class Dataset:
    """Признаки IP с разметкой.

    label      - 1 для синтетических ботов, 0 для реальных IP;
    rule_flag  - сработало ли надёжное правило (1, 3, 5, 6);
    kind       - тип синтетического бота или 'real';
    split      - train / val / test.
    """
    frame: pd.DataFrame

    def part(self, split, clean_only=False):
        df = self.frame[self.frame["split"] == split]
        if clean_only:
            # «чистые» - реальные IP, на которых не сработало ни одно надёжное
            # правило: на них измеряются ложные срабатывания
            df = df[(df["label"] == 1) | (df["rule_flag"] == 0)]
        return df

    def normal_train(self):
        df = self.part("train")
        return df[(df["label"] == 0) & (df["rule_flag"] == 0)]



def _split(index, strata, labels, test_size, seed):
    """train_test_split со стратификацией, устойчивый к маленьким группам.

    Редкие группы (на маленьких журналах) объединяются по метке; если и
    этого мало, разбиение делается без стратификации.
    """
    s = strata[index]
    s = s.where(s.map(s.value_counts()) >= 5, labels[index].astype(str))
    if s.value_counts().min() < 2:
        s = None
    return train_test_split(index, test_size=test_size, stratify=s,
random_state=seed)
```

```
def build_dataset(log, rule_results, bot_ips, config, val_size=0.2,
test_size=0.2):
    feats = extract_features(log, config.min_requests)
    df = feats.copy()
    df["label"] = df.index.isin(list(bot_ips)).astype(int)
    df["kind"] = [bot_ips.get(ip, "real") for ip in df.index]
    df["rule_flag"] = [int(bool(rule_results.rules_of(ip) & STRONG_RULES)) for
ip in df.index]
    df["rules"] = ["+".join(map(str, sorted(rule_results.rules_of(ip)))) for ip
in df.index]
    # стратификация по типу: в каждой выборке есть все виды ботов
    strata = df["kind"] + "_" + df["rule_flag"].astype(str)
    idx_train, idx_rest = _split(df.index, strata, df["label"], val_size +
test_size,
                                 config.random_state)
    idx_val, idx_test = _split(idx_rest, strata, df["label"],
                               test_size / (val_size + test_size),
config.random_state)
    df["split"] = "train"
    df.loc[idx_val, "split"] = "val"
    df.loc[idx_test, "split"] = "test"
    return Dataset(df)



def best_f1_threshold(y_true, scores):
    """Порог, при котором F1 на выборке максимален.

    precision_recall_curve считает ботами IP с оценкой >= thr, а детектор -
    с оценкой > порога. Поэтому порог ставится посередине между thr и
    ближайшей меньшей оценкой: сам IP с оценкой thr остаётся ботом.
    """
    scores = np.asarray(scores, dtype=float)
    prec, rec, thr = precision_recall_curve(y_true, scores)
    f1 = 2 * prec * rec / np.maximum(prec + rec, 1e-12)
    i = int(np.nanargmax(f1[:-1]))   # последняя точка кривой без порога
    lower = scores[scores < thr[i]]
    threshold = (thr[i] + lower.max()) / 2 if len(lower) else thr[i] - 1e-9
    return float(threshold), float(f1[i])



def train_detector(dataset, model_name, config, **model_params):
    """Обучить модель на чистых train-IP и подобрать порог на validation."""
    model_params.setdefault("random_state", config.random_state)
    det = AnomalyDetector(create_model(model_name, **model_params),
FEATURE_NAMES)
    normal = dataset.normal_train()
    t0 = time.perf_counter()
    det.fit(normal)
    fit_seconds = time.perf_counter() - t0
    val = dataset.part("val", clean_only=True)
    det.threshold, val_f1 = best_f1_threshold(val["label"], det.score(val))
    det.meta = {"val_f1": val_f1, "n_train": len(normal), "n_val": len(val),
                "fit_seconds": round(fit_seconds, 2), "config":
config.to_dict()}
    return det
```

Листинг А.4 – Итоговое решение (фрагмент lsdetector/pipeline.py)

```
class DetectionService:
    """Правила + модели. Модель-«решатель» (decision_model) определяет
    итоговое решение, остальные модели только добавляют оценки в отчёт."""
```

```
    def __init__(self, config=None, verifier=None, detectors=None,
decision_model=None):
        self.config = config or DetectorConfig()
        self.verifier = verifier or DnsVerifier(offline=self.config.dns_offline)
        self.detectors = detectors or {}
        self.decision_model = decision_model or (
            "autoencoder" if "autoencoder" in self.detectors else
next(iter(self.detectors), None))

    def run(self, log):
        t0 = time.time()
        rule_results = RuleEngine(default_rules(self.verifier),
self.config).run(log)
        t_rules = time.time() - t0

        verdicts = {}
        for ip, records in log.by_ip.items():
            if ip in rule_results.verified:
                continue
            hit = rule_results.hits.get(ip, {"rules": set(), "details": {}})
            verdicts[ip] = IPVerdict(ip, len(records), sorted(hit["rules"]),
hit["details"])

        if self.detectors:
            feats = extract_features(log, self.config.min_requests)
            feats = feats[feats.index.isin(list(verdicts))]
            if len(feats):
                for name, det in self.detectors.items():
                    scores = det.score(feats)
                    flags = scores > det.threshold
                    for ip, s, f in zip(feats.index, scores, flags):
                        verdicts[ip].scores[name] = float(s)
                        verdicts[ip].anomalous[name] = bool(f)
                if self.decision_model:
                    explained =
self.detectors[self.decision_model].explain(feats)
                    for ip, top in zip(feats.index, explained):
                        verdicts[ip].top_features = top

        for v in verdicts.values():
            strong = set(v.rules) & STRONG_RULES
            if strong:
                v.is_bot, v.reason = True, f"rule{primary_rule(strong)}"
            elif self.decision_model and v.anomalous.get(self.decision_model):
                v.is_bot, v.reason = True, f"model:{self.decision_model}"

        stats = {"lines": log.stats.get("lines", 0), "parsed":
log.stats.get("parsed", 0),
                 "skipped": log.stats.get("lines", 0) - log.stats.get("parsed",
0),
                 "seconds_rules": round(t_rules, 2),
                 "seconds_total": round(time.time() - t0, 2)}
        return DetectionReport(verdicts, rule_results, stats,
self.config.to_dict(),
                               list(self.detectors))
```

Листинг А.5 – Схема базы данных (api/schema.sql)

```
-- Схема БД истории анализов (SQLite).
```

```
-- Одна запись analyses - один загруженный журнал; ip_results - решения по его
IP.

PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS analyses (
    id               TEXT PRIMARY KEY,           -- короткий uuid
    filename         TEXT    NOT NULL,
    created_at       TEXT    NOT NULL,           -- ISO 8601, UTC
    decision_model   TEXT,                       -- autoencoder / iforest / NULL
    lines            INTEGER NOT NULL,
    unique_ips       INTEGER NOT NULL,
    suspicious_ips   INTEGER NOT NULL,
    params_json      TEXT    NOT NULL,           -- параметры запуска
(AnalyzeParams)
    summary_json     TEXT    NOT NULL,           -- сводка (Summary)
    thresholds_json  TEXT    NOT NULL            -- пороги моделей
);

CREATE TABLE IF NOT EXISTS ip_results (
    analysis_id    TEXT    NOT NULL REFERENCES analyses (id) ON DELETE CASCADE,
    ip             TEXT    NOT NULL,
    requests       INTEGER NOT NULL,
    rules          TEXT    NOT NULL,             -- номера правил через запятую
    is_bot         INTEGER NOT NULL CHECK (is_bot IN (0, 1)),
    reason         TEXT    NOT NULL,             -- rule1 ... /
model:autoencoder / ''
    scores_json    TEXT    NOT NULL,             -- {"autoencoder": 0.12, ...}
    anomalous_json TEXT    NOT NULL,
    top_features   TEXT    NOT NULL,             -- признаки через запятую
    description    TEXT    NOT NULL,
    details_json   TEXT    NOT NULL,
    PRIMARY KEY (analysis_id, ip)
);

CREATE INDEX IF NOT EXISTS idx_analyses_created ON analyses (created_at);
CREATE INDEX IF NOT EXISTS idx_ip_results_bot ON ip_results (analysis_id,
is_bot);
CREATE INDEX IF NOT EXISTS idx_ip_results_ip ON ip_results (ip);
```

ПРИЛОЖЕНИЕ Б

(обязательное)

Диаграммы классов и развёртывания

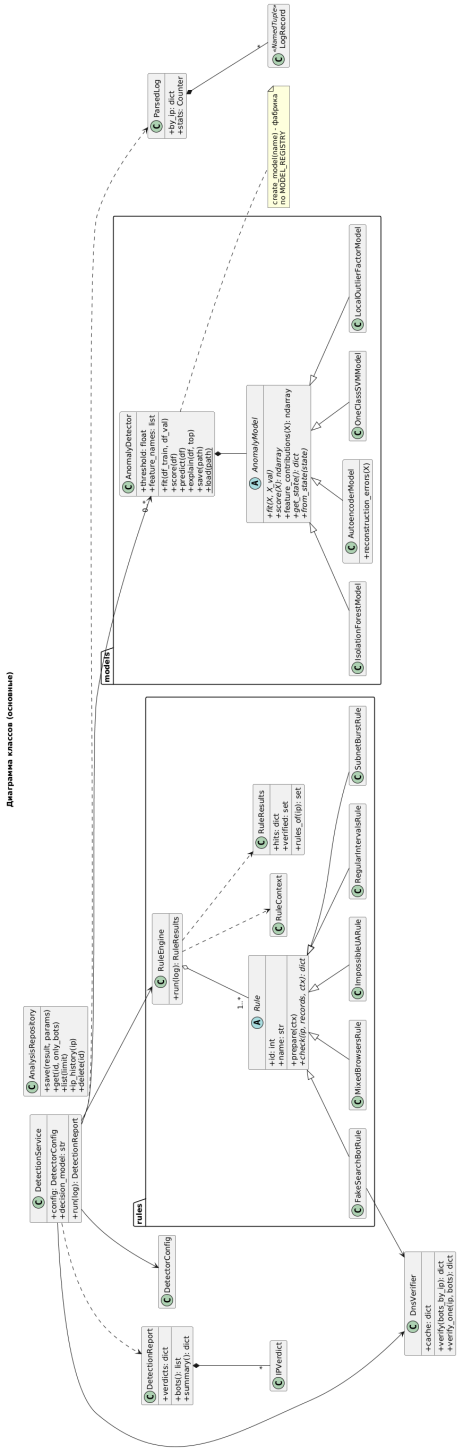


Рисунок Б.1 – Диаграмма основных классов (повёрнута)

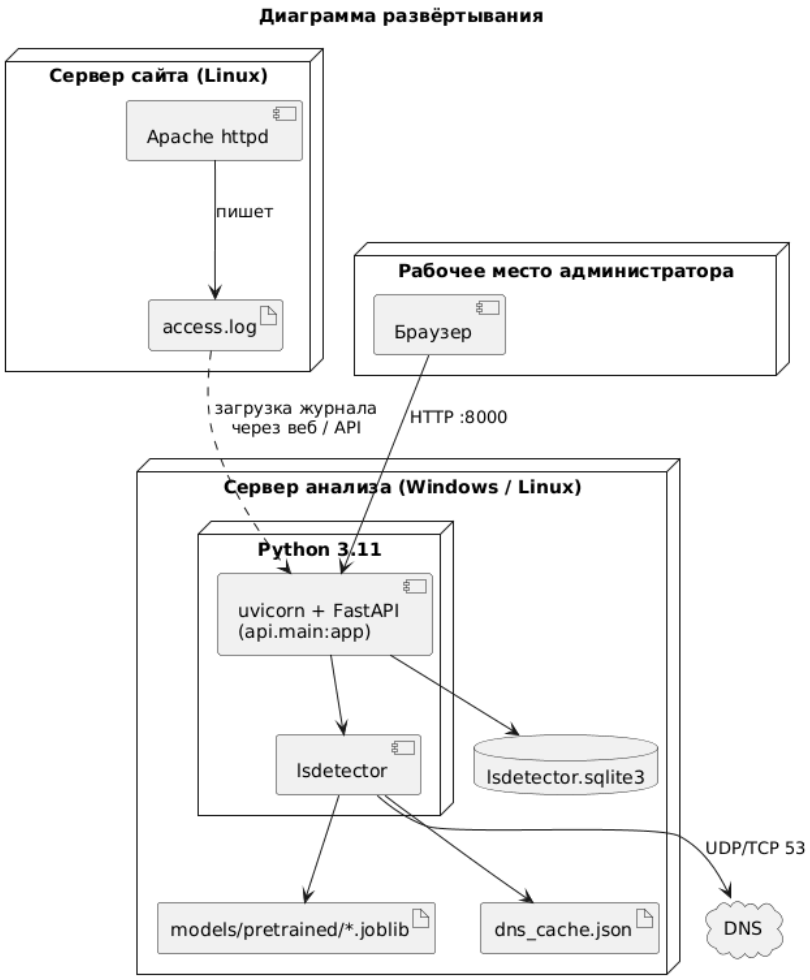


Рисунок Б.2 – Диаграмма развёртывания

ПРИЛОЖЕНИЕ В

(обязательное)

Дополнительные результаты тестирования

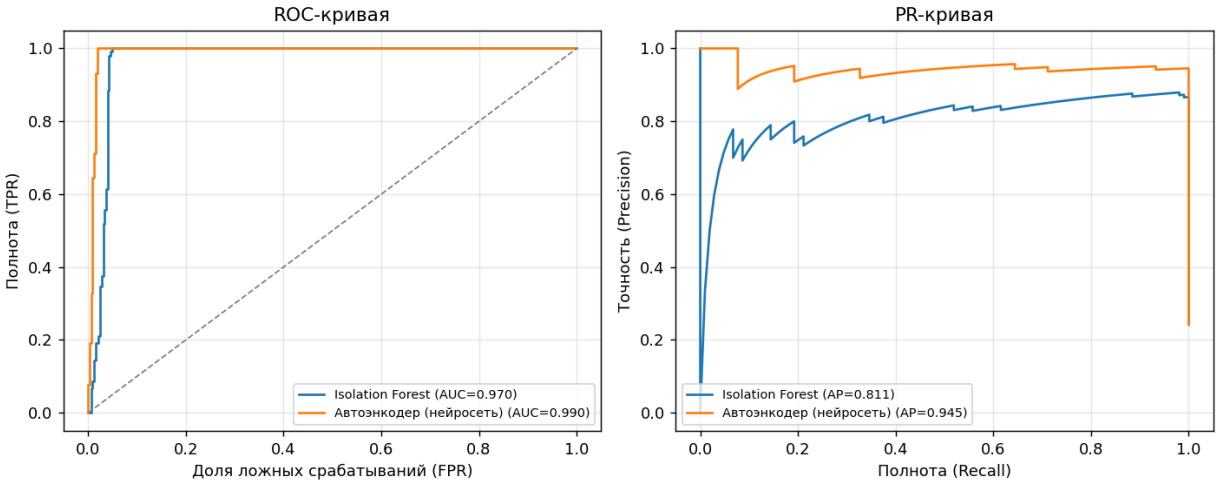


Рисунок В.1 – ROC- и PR-кривые моделей

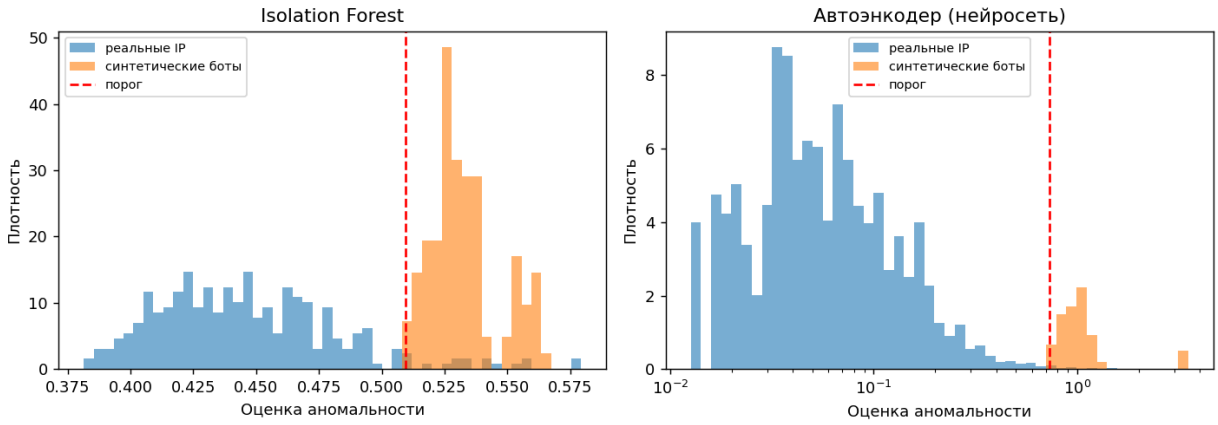


Рисунок В.2 – Распределение оценок аномальности

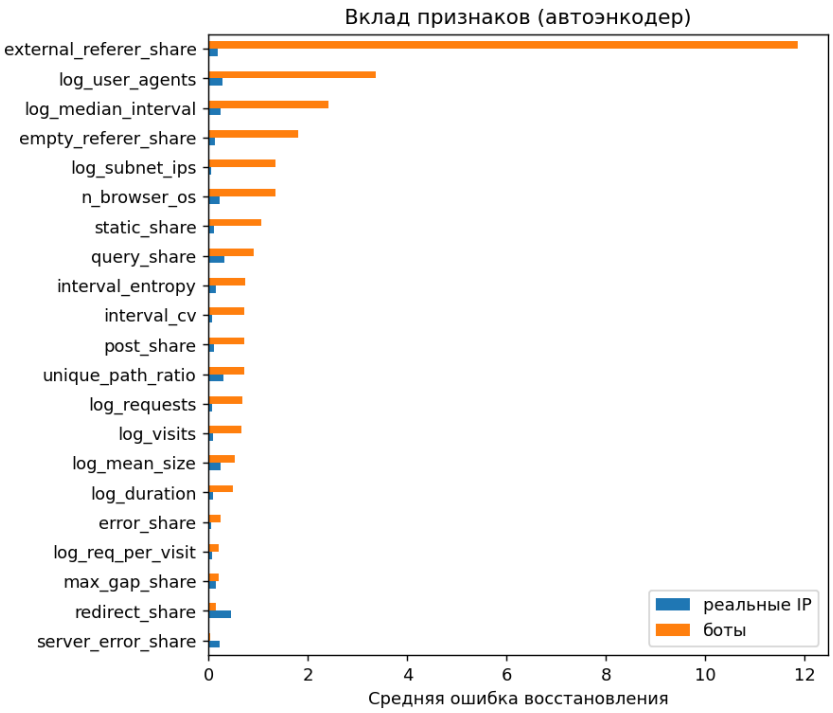


Рисунок В.3 – Средняя ошибка восстановления автоэнкодера по признакам



Рисунок В.4 – Документация REST API в Swagger

Таблица В.1 – Ошибки, найденные при разработке и тестировании

Ошибка	Как найдена	Исправление
Проверки User-Agent 3h–3m в прототипе не выполнялись (код после return)	ревью кода	перенесены, добавлены таблицы и тесты
Ложные срабатывания 3h на Chrome для iOS и приложениях Яндекса	прогон на реальном журнале	исключения в списке браузеров

Ошибка	Как найдена	Исправление
229 ложных срабатываний проверки 3i	прогон на реальном журнале	проверка помечена как слабая
Сбой стратифицированного разбиения на маленьком журнале	обучение на демо-журнале	объединение редких групп
Граничный IP выпадал из ботов (≥ против >)	расхождение счётчиков в интерфейсе	порог между соседними оценками, тест
Счётчик «выше порога» по округлённым оценкам	веб-интерфейс	подсчёт по флагу модели
Анализы в API выполнялись по очереди	нагрузочный тест	сужение блокировки кэша DNS
«database is locked» при нескольких процессах	нагрузочный тест	режим WAL, тайм-аут